

Fragmentación semántica y ensamblaje estocástico: un protocolo de inferencia descentralizada de modelos de lenguaje sobre nodos voluntarios no confiables

Sebastián A. Espinoza-Ulloa, Ph.D. Investigador independiente ORCID: [0000-0003-1497-356X](https://orcid.org/0000-0003-1497-356X) · GitHub: [@Sebastardito](https://github.com/Sebastardito) Correspondencia: sebas_saeu@hotmail.com

Nota sobre afiliación e independencia. Este trabajo se realiza íntegramente a título personal, como investigador independiente. El autor mantiene afiliaciones académicas separadas —Pontificia Universidad Católica del Ecuador (saespinozau@puce.edu.ec) y University of Saskatchewan (sebastian.espinoza@usask.ca)— ajenas a la materia de este trabajo. **Ninguna de las dos ha aportado financiación, materiales, recursos de cómputo, personal ni apoyo institucional a este proyecto, y no se reclama ni se implica respaldo institucional alguno.**

Trayectoria relevante. El autor es Doctor en Biología (University of Saskatchewan) en genómica de poblaciones de aves, y trabaja en secuenciación de genomas completos, llamado de variantes y ensamblaje *de novo*. El marco de ensamblaje genómico empleado en este documento procede de esa trayectoria; la sección 2.4 declara explícitamente dónde se sostiene la analogía y dónde no.

Versión 1.4 — 14 de agosto de 2026 Licencia de este documento: CC BY 4.0. Implementación de referencia: AGPL-3.0-or-later. Repositorio: <https://github.com/Sebastardito/Swarmbly-AI>

Propósito de este documento. Se trata de una divulgación *habilitante*, publicada de forma defensiva para constituir arte previo. Especifica el protocolo con detalle suficiente para que una persona con conocimientos en la materia pueda implementarlo, enuncia sus parámetros y su derivación, declara las hipótesis de las que depende su viabilidad y expone la evidencia que argumenta *en contra* con la misma prominencia que la que argumenta a favor. La sección 13 enumera los elementos divulgados en forma de reivindicaciones numeradas.

Resumen

La inferencia entre pares de modelos de lenguaje se ha abordado hasta ahora repartiendo el *modelo*: las capas o los tensores del transformer se distribuyen entre máquinas, y las activaciones intermedias atraviesan la internet pública en cada token generado. Esto sitúa el diseño de frente contra una brecha de ancho de banda de aproximadamente cinco órdenes de magnitud y una brecha de latencia de cuatro a cinco, entre la interconexión de centro de datos y los enlaces de última milla domésticos.

Este artículo especifica **Swarmbly**, un protocolo que distribuye el *problema* en su lugar. Un orquestador del lado del cliente, que es a su vez un modelo de lenguaje pequeño, descompone una petición en un grafo acíclico dirigido de microtarefas semánticas; cada microtarea se despacha una sola vez, de forma asíncrona, a un nodo voluntario que ejecuta un modelo pequeño completo (1-8 B de parámetros); los fragmentos devueltos —*contigs*, en el vocabulario del ensamblaje de genomas que el diseño toma prestado— se verifican, seleccionan y empalman localmente. La travesía de la red ocurre una vez por fragmento y por sesión, no una vez por capa y por token.

Hago cinco aportaciones. Primera, identifico el **presupuesto de contexto** S —el número de tokens de contexto compartido que acompañan a cada fragmento despachado— como un único escalar sobre el que cuatro requisitos de diseño tiran en direcciones contrapuestas: la coherencia del ensamblaje y la verificabilidad de los fragmentos crecen con S , la privacidad por descontextualización decrece con S , y la capacidad exigida al modelo trabajador plausiblemente decrece con S . La viabilidad del protocolo se reduce a si existe un valor de S que satisfaga los cuatro umbrales simultáneamente. Es una proposición falsable, y la enuncio como tal.

Segunda, descompongo el rendimiento del enjambre en **cobertura** (¿produce algún trabajador un fragmento aceptable?) y **conversión** (¿convierte el orquestador los fragmentos aceptables en un todo aceptable?), y argumento, a partir del registro publicado, que la cobertura escala con el número de nodos y la conversión no, lo que acota el valor marginal de un nodo adicional por la capacidad de selección del cliente e invierte la prioridad habitual de la ingeniería.

Tercera, especifico el protocolo de red, el algoritmo de ensamblaje, el esquema de verificación y las derivaciones de los parámetros con un detalle implementable.

Cuarta, publico un banco de pruebas de referencia que mide el **impuesto de coherencia** —la calidad que se pierde por la fragmentación y el reensamblaje— en función de S , junto con un criterio explícito de continuidad o abandono bajo el cual la arquitectura debería descartarse. **Esa medición ya se ha realizado.** Contra tres familias de modelos servidas localmente, el impuesto de coherencia decrece de forma monótona con S —24,1 %, 20,4 %, 16,1 %, 13,7 % a lo largo del rango barrido— y el criterio de abandono se cumple en tres categorías de tarea. En dos de ellas el impuesto es **negativo**: fragmentar el problema y reensamblarlo produjo una respuesta *mejor* que la línea base monolítica, hasta en un 9,0 %. La ejecución V3c acompañante no encuentra **ninguna relación entre el acuerdo entre réplicas y la calidad juzgada** ($r = -0,030$ sobre 597 unidades semánticas), lo que no respalda el mapa de confianza descrito más abajo; la sección 11.3

reporta ambos resultados completos, incluido por qué el segundo queda sin sustento y no refutado.

Quinta, introduzco un segundo eje de enrutamiento, ortogonal a la sensibilidad del contenido: un clasificador de privacidad del lado del cliente asigna a cada petición un **nivel** —malla abierta de voluntarios, *enjambre de confianza* permissionado cuya pertenencia es una lista blanca criptográfica de claves públicas bajo TLS mutuo, o ejecución puramente local— y el mismo protocolo y el mismo cliente se ejecutan en los tres. Esto es lo que hace la arquitectura desplegable allí donde un voluntario anónimo no puede ser lícitamente un encargado del tratamiento, y separa dos papeles que el número de réplicas k venía desempeñando a la vez: la defensa frente a trabajadores deshonestos, que la lista blanca elimina, y las réplicas independientes que el mapa de confianza necesita, que no.

El cómputo ya existe. Las plataformas de computación voluntaria agregan hoy del orden de 700.000 dispositivos activos, cuatro millones de núcleos de CPU y 560.000 GPU a un rendimiento medio de 93 PetaFLOPS, y lo hacen desde una base de participantes que lleva dos décadas *decreciendo*, lo que convierte esa cifra en un suelo y no en un techo. Lo que falta no es silicio. Es un protocolo bajo el cual ese silicio pueda servir inferencia de modelos de lenguaje sin que sus propietarios cedan el control y sin una interconexión de centro de datos.

Swarmby es ese protocolo, y su consecuencia arquitectónica central es que **la barrera para servir IA deja de ser el capital y pasa a ser la participación**.

La descentralización aporta además una capacidad que la centralización no puede replicar. Cuando k réplicas de una microtarea las producen nodos que ejecutan familias de modelo *distintas* y se alinean entre sí, el acuerdo por unidad entre ellas es una señal medible, y el protocolo devuelve un **mapa de regiones de baja confianza** junto a cada respuesta: el análogo directo de la calidad por base en un ensamblaje de genoma. Un proveedor monolítico que ejecuta un solo modelo no tiene nada que alinear. Es una propiedad epistémica que emerge *de* la distribución en vez de sacrificarse a ella.

Enuncio con precisión lo que no afirmo —paridad de latencia, contexto ilimitado, confidencialidad criptográfica, un beneficio de carbono demostrado— en la sección 1.4, y dedico la sección 12 a las limitaciones y los resultados negativos, porque una especificación que puede demostrarse errónea vale más que una promesa que no puede comprobarse.

Palabras clave: inferencia descentralizada · sistemas entre pares · descomposición de prompts · modelos de lenguaje pequeños · computación verificable · computación voluntaria · ensamblaje de genomas

1. Introducción

1.1 La concentración está en el capital, no en el conocimiento

La capacidad de construir modelos de lenguaje ya no es escasa. Los pesos de los modelos, las recetas de entrenamiento y los motores de inferencia se publican abiertamente y mejoran cada mes. Lo que sigue siendo escaso —y lo que concentra el poder— es el capital necesario para *operarlos* a escala: los aceleradores, los edificios, los contratos de energía y la interconexión.

Esa concentración tiene una forma medible. Los centros de datos consumieron 415 TWh en 2024, en torno al 1,5 % de la electricidad mundial, con proyecciones de 945 TWh para 2030 [83]. Las instalaciones hiperescala de Estados Unidos se abastecen de redes eléctricas medidas en 545 gCO₂/kWh frente a una media nacional de 370 g [84]. Son las cifras de una industria cuya vía de crecimiento pasa por la construcción, y la construcción solo está al alcance de quien puede financiarla.

La consecuencia es estructural antes que conspirativa: una tecnología cuyo *conocimiento* es público se vuelve, en la práctica, controlable por quien pueda costear el *hardware*. La apertura de los pesos no democratiza una capacidad cuya operación cuesta cientos de millones de dólares.

Y sin embargo el hardware ya existe, distribuido y ocioso. La plataforma insignia de computación voluntaria agrega aproximadamente 700.000 dispositivos activos, 4 millones de núcleos de CPU y 560.000 GPU a 93 PetaFLOPS de media [47], y lo hace desde una base de participantes que ha pasado de cerca de un millón a unos doscientos mil en dos décadas. Esa cifra es un *suelo*, extraído de un nicho en declive, y no una proyección de lo que podría movilizar un protocolo convincente. Medido a nivel del nodo individual, las GPU domésticas ociosas sirven inferencia de LLM a 0,111–0,149 \$ por millón de tokens en una RTX 4090, al 62–78 % del rendimiento de una H100 por aproximadamente la mitad del coste [49].

La capacidad de inferencia ociosa del mundo no es una hipótesis. La pieza que falta es un protocolo bajo el cual pueda usarse, y la razón de que aún no exista tal protocolo es una restricción física que la siguiente sección enuncia con exactitud.

1.2 La restricción física y el replanteamiento

Cualquier arquitectura de inferencia distribuida de modelos de lenguaje sobre hardware doméstico queda decidida, antes de elegir algoritmo alguno, por una sola medición. Una NVIDIA H100 SXM mueve 900 GB/s por GPU sobre NVLink; Quantum-2 InfiniBand ofrece 400 Gb/s por puerto y 51,2 Tb/s agregados por conmutador. El ancho de banda de subida doméstico típico es del orden de 60 Mbps. La razón es de aproximadamente 120.000× frente a NVLink y 6.700×

frente a InfiniBand. La latencia intranodo de NVLink es submicrosegundo y la de InfiniBand de microsegundos de un dígito, frente a 30–170 ms de tiempo de ida y vuelta de área amplia: **cuatro a cinco órdenes de magnitud** [22, 23, 24].

Este único hecho parte el espacio de diseño con nitidez. Las arquitecturas que requieren comunicación *por token* quedan empujadas contra la brecha en cada paso de la generación, mientras que las que la cruzan *una vez por unidad de trabajo* no lo están, y todo lo demás en este artículo se sigue de elegir la segunda clase.

El comportamiento medido de la primera clase es coherente con la predicción. Petals —la implementación de referencia de la inferencia con paralelismo de tubería sobre internet— sirve Llama-2-70B en tres T4 a 2,29 pasos/s sobre un enlace de 1 Gbit/s con RTT inferior a 5 ms, y cae a 1,57 pasos/s a 100 Mbit/s y 100 ms: una pérdida del 31 % atribuible únicamente a la red. Un enjambre geodistribuido real de catorce servidores heterogéneos alcanza 0,83 pasos/s [1, 2]. Los análisis de esquemas de paralelismo de modelo a latencias de internet pública concluyen que el paralelismo de tubería es el *único* arreglo de paralelismo de modelo viable —es el que menos comunica— y que el microlotado asíncrono no ayuda, porque la decodificación está limitada por el movimiento de la caché KV y no por el cómputo [25].

La conclusión que extraigo no es que el paralelismo de tubería estuviera mal implementado. Es que es la respuesta correcta a la pregunta equivocada.

Swarmblly formula una pregunta distinta: en lugar de *cómo ejecutar un modelo grande en muchas máquinas*, **cómo ejecutar muchos modelos pequeños completos sobre un problema grande**.

No son variantes de la misma idea. Repartir un modelo crea una cadena en la que el nodo k no puede empezar hasta que el nodo $k-1$ termina, y en la que cada token recorre de nuevo la cadena. Repartir un problema crea un conjunto —más precisamente un orden parcial— en el que las subtarefas independientes progresan de forma concurrente y cada una cruza la red una sola vez. La primera está acotada por $\sum_i (t_{\text{compute},i} + t_{\text{net},i})$; la segunda por $\max_i (t_{\text{compute},i} + t_{\text{net},i})$ más el ensamblaje local. La afirmación estructural es que el segundo es el régimen en el que el hardware voluntario puede participar en absoluto.

El vocabulario de diseño se toma deliberadamente del ensamblaje shotgun de genomas. Una petición se fragmenta en *reads*; cada respuesta devuelta es un *contig*; los contigs adyacentes se unen por *solapamiento* y, donde discrepan, por *consenso*; el plan que los ordena es un *scaffold*. Somos explícitos en la sección 2.4 sobre lo que esta analogía autoriza y lo que no: aporta un vocabulario, un conjunto de modos de fallo y una advertencia genuinamente transferible. No aporta un algoritmo transferido, y no afirmo tal cosa.

1.3 Qué hace posible esto

La primera medición ya está hecha, y la predicción central se sostuvo: el impuesto de coherencia decrece de forma monótona con el presupuesto de contexto, y en tres categorías de tarea queda por debajo del umbral de abandono que se fijó antes de que existiera dato alguno —en dos de ellas produciendo una respuesta *mejor* que la línea base monolítica (sección 11.3). Una ejecución, a una escala, sobre ocho prompts —uno de los cuales no produjo una línea base utilizable— no convierte un protocolo en probado, y la sección 11 sigue enunciando la medición bajo la cual concluiría que el diseño falla. Lo que sí significa es que el núcleo falsable de la sección 4 sobrevivió a su primer contacto con la evidencia, y que se siguen cuatro cosas que hoy no están disponibles.

1. Capacidad de servicio sin poseerla. Un participante aporta una máquina que ya existe y que ya consume energía cuando está ociosa. El requisito de entrada es un modelo pequeño completo, no un fragmento de uno grande, lo que sitúa el conjunto de hardware alcanzable órdenes de magnitud por encima de lo que pueden alcanzar los esquemas de paralelismo de tubería. La capacidad escala entonces con la *participación* y no con el gasto de capital: una curva de crecimiento que ningún operador centralizado puede igualar, porque la suya está acotada por lo que puede construir y financiar.

2. Un mapa de confianza que la centralización no puede producir estructuralmente: un mecanismo, todavía no un beneficio demostrado. La sección 8.4b lo desarrolla. En un borrador anterior de este artículo se describía como la propiedad de cara al usuario más inmediatamente valiosa de la arquitectura; la primera medición (sección 11.3) no respalda esa descripción y se ha retirado. Lo que queda es un mecanismo cuyo valor está sin medir. Como una microtarea la responden k nodos que ejecutan familias de modelo *distintas*, las respuestas pueden alinearse entre sí y el acuerdo puntuarse por unidad semántica. Las regiones donde modelos independientes convergen se reportan como tales; las regiones donde divergen se exponen como de baja confianza, exactamente igual que un ensamblador reporta la calidad por base en lugar de una secuencia uniformemente confiada. **Un proveedor que ejecuta un solo modelo no tiene nada que alinear.** La redundancia que la descentralización exige resulta producir una señal que la centralización no puede obtener a ningún precio. **Que esa señal contenga información sobre la corrección es una cuestión distinta, y el primer intento de medirlo salió plano** (sección 11.3). El mecanismo es real; su utilidad está sin demostrar, y el experimento que zanjaría la cuestión se especifica en la sección 11.4.

3. Un contexto acotado por la máquina del usuario y no por la decisión de producto de un proveedor. La fragmentación reubica el límite de contexto: pasa de una ventana fija fijada por un proveedor a una función del tiempo y de la memoria de ensamblaje del cliente. Con ensamblaje jerárquico, la memoria de trabajo requerida crece logarítmicamente con el volumen total, de modo que el techo práctico para una máquina personal moderna queda muy por encima de lo que agotaría un usuario individual y, a diferencia de la ventana de un proveedor, sube cuando el usuario mejora su equipo y no cuando cambia un plan de precios.

4. Un sustrato que puede auditarse en vez de confiarse. El protocolo, el cliente, el software de nodo y la licencia son públicos. La cuota de tráfico servida por nodos ancla operados por la fundación se publica (sección 10.4). La contabilidad energética se publica contra un estándar público (sección 10.3). La degradación de coherencia se devuelve con cada respuesta en vez de ocultarse (sección 8.6). Ninguna de estas cosas es una cortesía; cada una es un requisito de conformidad de la especificación, y una implementación que las omita no es conforme.

En conjunto, describen una distribución distinta del control sobre una tecnología de propósito general, no una forma más barata de comprar lo mismo. No voy a presentar eso como una afirmación modesta, porque no lo es: si el presupuesto de contexto se sostiene a escala, la condición previa para *servir* una tecnología de propósito general deja de ser un centro de datos y pasa a ser un portátil y un protocolo. Que esa redistribución sea alcanzable es una cuestión empírica, y el resto de este artículo está escrito para hacerla responsable en vez de retórica.

1.4 Alcance de las afirmaciones

Cuatro afirmaciones que un lector podría esperar aquí están deliberadamente ausentes, y la sección 12 desarrolla cada una en detalle.

No afirmo **paridad de latencia**: la decodificación especulativa en un solo nodo ya entrega 2-3× con una demostración de que la distribución de salida se preserva [13], y ningún esquema de fragmentación compite con eso en velocidad. La comparación que importa es otra: para un usuario que no tiene el hardware para ejecutar en absoluto un modelo capaz, el eje relevante no es *más rápido o más lento*, sino *posible o imposible*.

No afirmo **contexto ilimitado**, solo un límite reubicado y mucho más alto (sección 1.3, punto 3).

No afirmo **confidencialidad criptográfica**. La fragmentación no es cifrado, la sección 9 expone los ataques que zanján la cuestión, y el protocolo encamina el trabajo sensible a la ejecución local o a hardware atestiguado en vez de pretender otra cosa.

No afirmo un **beneficio ambiental demostrado**. El argumento del carbono incorporado es sólido y el operativo es condicional; la sección 10.3 enuncia ambos, junto con la medición que me comprometo a publicar muestre lo que muestre.

Enunciar esto con claridad no cuesta nada que fuera real, y es lo que permite leer las afirmaciones de la sección 1.3 como ingeniería y no como publicidad.

1.5 Aportaciones y estructura

La sección 2 revisa el estado del arte y la sección 3 enuncia los principios de diseño. La sección 4 desarrolla el **presupuesto de contexto**, la aportación conceptual central del artículo, y la sección 5 formaliza la tesis del **enjambre de modelos pequeños** y enuncia la descomposición cobertura/conversión. La sección 6 presenta la arquitectura, la 7 el protocolo de red y la 8 los algoritmos. La sección 9 cubre privacidad, verificación, niveles de privacidad y nodos adversarios, y la sección 10 esboza economía y gobernanza. La sección 11 describe el banco de pruebas de referencia y el protocolo de evaluación, incluido el criterio bajo el cual abandonaría el diseño. La sección 12 enumera limitaciones y resultados negativos, y la sección 13 declara los elementos divulgados a efectos de arte previo.

2. Antecedentes y trabajo relacionado

2.1 Inferencia descentralizada por partición del modelo

Petals [1, 2] distribuye bloques contiguos de capas del transformer entre voluntarios; los clientes retienen los embeddings localmente y encaminan las activaciones a través de una cadena de servidores. Lo trato como el pionero del campo y no como un competidor: demostró que la inferencia entre pares sobre la internet pública es siquiera posible, que es la precondition de este trabajo. Swarmby no mejora el paralelismo de tubería; renuncia a usarlo, y esa divergencia es una diferencia de estrategia y no de calidad. La última versión de Petals data de septiembre de 2023 [3]. Hivemind y el paralelismo SWARM [4, 5] abordan el entrenamiento tolerante a fallos sobre dispositivos heterogéneos poco fiables con la misma premisa subyacente: el modelo es la unidad de distribución.

Bittensor [6, 7] añade una capa de incentivos, con un mecanismo de consenso cuya regularización basada en conectividad se describe como resistente a la colusión de hasta el 50 % del peso de la red, formulación que, leída con atención, presupone un anclaje de confianza.

2.2 Entrenamiento descentralizado sobre enlaces lentos

El subcampo que más ha avanzado es el entrenamiento, y avanzó atacando el volumen de comunicación en vez de la topología. DiLoCo iguala la optimización totalmente síncrona comunicando 500× menos [8]. OpenDiLoCo entrenó a través de dos continentes con un 90-95 % de utilización de cómputo [9]. INTELLECT-1 entrenó un modelo de 10 B de parámetros sobre 1 T de tokens en hasta 14 nodos concurrentes en tres continentes con 30 contribuyentes independientes y una reducción de ancho de banda de 400× [10]. Subspace/Protocol Models reportan igualar la convergencia del paralelismo de modelo de centro de datos a 80 Mbps frente a 100 Gbps [11].

La lección que Swarmby extrae es metodológica: el problema del ancho de banda cede ante la compresión y la

asincronía. Un protocolo que reinvente esto en vez de adoptarlo está malgastando esfuerzo.

2.3 Paralelismo a nivel de tarea

Skeleton-of-Thought (SoT) [12] es el precedente directo de la descomposición de un *prompt*: un prompt de esqueleto produce una lista de puntos, cada uno expandido de forma independiente y en paralelo. Reporta hasta 2,39× de aceleración y —esto importa más— reporta su propio daño: la calidad mejora en preguntas de conocimiento, genéricas, de sentido común, de juego de rol y contrafácticas, y se degrada en matemáticas, programación, redacción y estimación de Fermi; en la métrica de coherencia, SoT «no es peor que la generación normal alrededor del 60 % de las veces», lo que equivale a decir que sí es peor aproximadamente el 40 % de las veces. Los autores enuncian la causa estructural sin paliativos: «SoT actualmente ignora las dependencias entre puntos».

Su respuesta no fue defender el método, sino condicionarlo. SoT-R [12] añade un router que decide, pregunta a pregunta, si descomponer siquiera; basta un router RoBERTa de 120 M entrenado, y se entrena con una pérdida de Tversky precisamente para penalizar los falsos positivos, codificando la asimetría de que fragmentar indebidamente es peor que negarse indebidamente a hacerlo.

Los descendientes refinan la idea. APAR [16] hace que el modelo planifique sus propias ramas paralelas. PASTA [17] aprende un lenguaje de anotación para tramos semánticamente independientes y reporta aceleraciones de media geométrica de 1,21–1,93× con un delta de tasa de victoria controlada por longitud de +2,2 % a −7,1 %, la curva velocidad/calidad más honesta publicada en esta familia. Plato/ASGD [18] sustituye la lista plana por un **grafo de dependencias** sobre los subproblemas y reporta una ganancia de rendimiento del 68 % con una tasa neta de victoria de calidad del 90 % frente a SoT. Hogwild! Inference [19] adopta la vía opuesta: trabajadores concurrentes que comparten una caché KV viva, y encuentra que los modelos de razonamiento actuales hacen esto sin ajuste fino.

ParallelBench [20] aporta la teoría: el supuesto de independencia condicional que subyace a la generación paralela «degrada inevitablemente la calidad de la generación cuando las dependencias son fuertes». Tran y Kiela [21] dan la versión teórico-informacional vía la desigualdad de procesamiento de datos, y encuentran que el agente único es el mejor o está estadísticamente empatado en todos los presupuestos de tokens de razonamiento por encima del más pequeño.

Lo que falta en esta literatura es mi objeto: **nadie ha combinado la descomposición a nivel de prompt con el despacho a nodos voluntarios no confiables**. Esa intersección, y no ninguna de sus dos mitades, es lo que este documento divulga.

2.4 Ensamblaje de genomas: qué autoriza la analogía

La estadística de cobertura de Lander-Waterman [26] da, para un genoma de longitud G muestreado por N clones de longitud L con fracción mínima de solapamiento detectable θ :

$c = L \cdot N / G$		(redundancia de cobertura)
$P(\text{base no cubierta})$	$= e^{(-c)}$	
$E[\text{\# islas aparentes}]$	$= N \cdot e^{(-c \cdot \theta)}$	
$E[\text{\# clones por isla}]$	$= e^{(c \cdot \theta)}$	

con la simplificación familiar $\theta \rightarrow 0$ $E[\text{contigs}] = N \cdot e^{(-c)}$ de la que se sigue la regla de la «cobertura 8×»: $e^{(-8)} \approx 0,034$ % de bases sin cubrir.

Este modelo sí se transfiere, pero solo tras una corrección que las versiones anteriores de este trabajo hacían mal.

La diferencia relevante entre el ensamblaje de genomas y el ensamblaje de texto **no** es que la secuencia objetivo sea *conocida*. En el ensamblaje *de novo* no existe referencia alguna: la secuencia se recupera por alineamiento, por probabilidad y por comprobaciones de plausibilidad biológica sobre el consenso. Una versión anterior de este artículo decía «preexistente» de un modo que implicaba «conocida», y eso era sencillamente un error.

La diferencia real es más estrecha y más útil. En genómica existe **una única molécula física de la que cada read es una muestra**. Esa unicidad es lo que garantiza que dos solapamientos verdaderos sean reconciliables: ambos reads provienen del mismo objeto. En la generación de texto libre no existe tal objeto garante: dos nodos que escriben sobre el mismo subtema no están muestreando nada común; están *creando* de forma independiente contenido que puede coincidir o no.

Existe, por separado, un sustrato formal compartido real: el problema de la supercadena común más corta subyace tanto al ensamblaje de genomas como a la reconstrucción de textos [27]. Lo que no existe es un algoritmo transferido, y la dirección histórica de la técnica corre en sentido contrario: la computación distribuida y de alto rendimiento se ha aplicado *al* ensamblaje, no se ha derivado *de* él [28].

Pero el objeto garante puede fabricarse. Si el plan $\mathcal{D}$ y el contrato global Γ se fijan *antes* de que ocurra generación alguna y se tratan como la referencia —una secuencia semántica que cada fragmento muestrea—, entonces vuelve a existir un objeto subyacente común, y la estadística de cobertura pasa a ser aplicable. La sección 5.4.1 desarrolla esto, y convierte la analogía de convención de nomenclatura en derivación.

Queda una transferencia genuina de la literatura de ensamblaje, y es una advertencia. En el ensamblaje de de Bruijn, una repetición más larga que k colapsa en un único nodo del grafo; el camino euleriano deja de ser único, y el número de reconstrucciones válidas crece combinatoriamente con el número de repeticiones [29]. Veinte años de práctica

establecieron la consecuencia: **las repeticiones, no la cobertura, son la restricción vinculante** [30, 31]. Añadir profundidad no resuelve una repetición. Traducido: aumentar la redundancia no arregla un ensamblaje cuyos fragmentos son semánticamente ambiguos unos respecto de otros, y los modos de fallo que importan —quimeras, colapsos, malos ensamblajes [32]— son estructurales antes que estadísticos.

3. Principios de diseño

- P1 — Cruzar la red una vez por unidad de trabajo.** El único argumento de rendimiento defendible al alcance de una red voluntaria.
- P2 — El orquestador puede negarse.** Un sistema que fragmenta toda petición es estrictamente peor de lo que era SoT en 2023, porque SoT incorporó un router. Fragmentar es una decisión con una función de coste asimétrica, no un comportamiento por defecto.
- P3 — Modelar las dependencias explícitamente.** Los planes son grafos acíclicos dirigidos. El paralelismo es la anchura de un nivel, no el tamaño del conjunto de tareas.
- P4 — Seleccionar antes que sintetizar.** Donde existan varios fragmentos candidatos, elegir uno y empalmarlo. Reescribir solo donde una costura falla realmente. La sección 5.3 da la evidencia; es el hallazgo más contraintuitivo de la literatura que he revisado.
- P5 — Calibrar todo umbral.** Sin cortes de coseno fijos, sin tasas de redundancia asumidas. Los umbrales se derivan de datos etiquetados por modelo y por dominio, con objetivos asimétricos, y se rederivan cada vez que cambia el modelo de embeddings.
- P6 — Reportar el impuesto.** Todo ensamblaje devuelve una auditoría de coherencia. Un protocolo que oculta su propia degradación no puede evaluarse, y no será digno de confianza.
- P7 — Verificar barato o no verificar.** Una verificación que cuesta una fracción significativa de la inferencia destruye la economía. La sección 9.3 selecciona esquemas con sobrecostes del orden del 1 %.
- P8 — Encaminar por sensibilidad, no fingir que se cifra.** La confidencialidad es una decisión de encaminamiento con tres carriles, no una propiedad atribuida a la fragmentación.

4. El presupuesto de contexto

Esta sección enuncia la restricción central del artículo. Es lo que el desarrollo previo de este proyecto —y, hasta donde alcanzo a ver, la literatura circundante— deja implícito.

4.1 Definición

Sea una petición P descompuesta en microtareas $\tau = \{\tau_1 \dots \tau_N\}$ con DAG de dependencias $D = (V, E)$. Cada paquete despachado es

$$K_i = (\Gamma, \sigma(R_j : (\tau_j \rightarrow \tau_i) \in E), \tau_i)$$

donde Γ es el *contrato global* —objetivo, audiencia, registro, formato de salida, longitud objetivo, vocabulario prohibido, identificador de sesión— y $\sigma(\cdot)$ resume los resultados de los predecesores de τ_i .

Defino el **presupuesto de contexto**

$$S = |\Gamma| + E[|\sigma(\cdot)|] \quad (\text{tokens de contexto compartido por paquete})$$

y la **tasa de redundancia contextual**

$$\rho = (\sum_i |K_i|) / |P| \approx 1 + N \cdot S / |P|$$

ρ es lo que paga el operador; S es lo que el operador elige.

4.2 La tensión a cuatro bandas

Cuatro requisitos son funciones de S , y no concuerdan entre sí:

Requisito	Comportamiento en S	Mecanismo
Coherencia del ensamblaje	crece	Los trabajadores comparten las decisiones que hacen compatibles los fragmentos. En ausencia de contrato, un trabajador representa la escena en un registro y otro en un segundo, y el cliente hereda partes incompatibles [33]
Verificabilidad del fragmento	crece	Un verificador no puede juzgar si un fragmento es fiel a una especificación que no se le dio

Privacidad por descontextualización	decrece	Γ es el objetivo, la audiencia y las restricciones de la sesión. Un nodo que posee Γ posee la forma de la petición
Capacidad exigida al trabajador	plausiblemente decrece	El contexto suministrado en el prompt sustituye al conocimiento almacenado en los parámetros; enunciado como hipótesis en la sección 5.4, no como resultado

Y ρ , y por tanto el coste, crece de forma aproximadamente lineal en $N \cdot S$.

4.3 El núcleo falsable

Proposición (Presupuesto de contexto). Swarmbly es viable si y solo si existe un presupuesto de contexto S^* que satisfaga simultáneamente: un impuesto de coherencia por debajo de la tolerancia de la aplicación; una cota de fuga por debajo de la tolerancia del usuario para el carril de sensibilidad aplicable; una exactitud de verificación por encima del requisito de seguridad del protocolo; y una exigencia de capacidad del trabajador satisfecha por modelos pequeños de mercado, todo ello a una ρ cuyo coste se mantenga por debajo del valor de la capacidad agregada.

Este es el proyecto entero enunciado como una única afirmación comprobable, y es la razón por la que la implementación de referencia mide una curva en lugar de demostrar un sistema. Cada pata tiene su propio experimento: coherencia en V0 (sección 11.2), verificación en V3, fuga en una auditoría de privacidad dedicada, sustitución de capacidad en el protocolo H2 de la sección 5.4.

También predice algo útil. Como S es compartido por las cuatro, **cualquier mejora que eleve la coherencia por token de contexto vale más que una mejora que eleve la coherencia por token de salida**: compra progreso en privacidad y en coste a la vez. Esto convierte a la compresión del contrato, y no a la calidad de los fragmentos, en la dirección de investigación de mayor apalancamiento del diseño. No lo esperaba cuando empecé, y es el tipo de predicción que hace que valga la pena enunciar formalmente este marco.

4.4 Por qué la formulación anterior era inadecuada

Una versión anterior de este diseño especificaba un objetivo fijo de redundancia ($C_{\text{sem}} > 1,2$, «20 % de redundancia intencional») derivado por analogía de Lander-Waterman, y un umbral de costura fijo ($\tau_{\text{sem}} = 0,85$) sobre la similitud coseno de embeddings.

Ambos se retiran. El primero se retira por las tres razones de la sección 2.4 y porque el paso de una razón a un porcentaje de redundancia solo se sostiene si todo el exceso de longitud es flanco, lo que deja de ser cierto en el momento en que se introduce un contrato global. El segundo se retira porque el espacio de embeddings contextuales es anisótropo —palabras elegidas al azar ya exhiben una similitud coseno media alta [34]—, porque la similitud coseno en modelos regularizados puede ser «arbitraria y por tanto carente de significado», determinada por el esquema de regularización y no por la semántica [35], porque ningún modelo de embeddings domina en todos los tipos de tarea [36], y porque la biblioteca de referencia para esta operación recomienda deliberadamente **ningún umbral en absoluto** y advierte de que la similitud es asimétrica [37].

Se sustituyen por medición. ρ se barre; τ se calibra a partir de pares etiquetados de costura y no-costura bajo un objetivo asimétrico (sección 8.5). Donde la formulación anterior afirmaba constantes, esta especifica procedimientos para obtenerlas.

5. Inteligencia de enjambre con modelos pequeños

5.1 La tesis

Swarmbly prescinde por completo del modelo monolítico. Ningún participante posee un fragmento de una red de 70 B o 400 B. En su lugar, los nodos trabajadores ejecutan modelos de lenguaje pequeños *completos e independientes* —típicamente de 1-8 B de parámetros, cuantizados, en GPU domésticas, portátiles de memoria unificada o CPU multinúcleo—, y cada uno de ellos recibe una microtarea atómica y descontextualizada y la responde de forma aislada. El cliente ejecuta también un modelo pequeño, pero su competencia es otra: no el conocimiento del mundo, sino la *lógica* y la *sintaxis*, es decir, comprender la petición, planificar su descomposición y suturar los fragmentos devueltos en un todo coherente.

La afirmación es que la capacidad avanzada no tiene por qué residir en una única red grande, sino que puede ser el resultado aritmético de coordinar muchas pequeñas: la respuesta emerge, como lo hace un genoma, solo en el ensamblaje.

Es una afirmación fuerte. También está en parte respaldada, en parte sin respaldo, y en parte es falsa tal como suele enunciarse. Esta sección separa las tres partes.

5.2 Cobertura y conversión

Propongo descomponer el rendimiento del enjambre en dos factores independientes.

La **cobertura** C es la probabilidad de que *al menos una* respuesta de trabajador a una microtarea dada sea aceptable, y la **conversión** V es la probabilidad de que el orquestador, dado que existen fragmentos aceptables, los seleccione y

ensamble en un todo aceptable.

Para un plan de N microtareas con cobertura por tarea C_i y un factor de conversión global V :

$$Q_{\text{system}} \approx V \cdot \Pi_i C_i$$

El producto sobre i es el término incómodo —es la razón por la que N no puede crecer libremente—, pero el factor que decide la arquitectura es V .

La cobertura escala con el enjambre. La conversión no.

La evidencia de la primera mitad es sólida. El muestreo repetido eleva la cobertura de forma log-lineal a lo largo de cuatro órdenes de magnitud del número de muestras: en SWE-bench Lite con DeepSeek-Coder-V2-Instruct, el 15,9 % con una muestra sube al 56 % con 250 muestras, superando un estado del arte del 43 % con muestra única [38]. Más nodos significa genuinamente que existen más fragmentos correctos en algún lugar del enjambre.

La evidencia de la segunda mitad es igual de sólida y suele pasarse por alto. El mismo trabajo afirma que «el voto mayoritario y los modelos de recompensa se estancan más allá de varios centenares de muestras»: la cobertura sigue subiendo y la *capacidad de convertirla en valor* se satura [38]. La selección basada en juez sobre equipos diversos alcanza una tasa de victoria del 81 % frente a una línea base de modelo único, mientras que los equipos homogéneos alcanzan el 51,2 % —el azar— y producen el 100 % de empates en 756 veredictos bajo juicio desacoplado [39]. Y en el estudio más próximo a la arquitectura propia de Swarmby, un sistema multiagente de 8 B empató con un agente único de 32 B con herramientas en GAIA (23,0 frente a 23,0) y lo supera en AIME (55,0 frente a 45,0), ejecutándose 4,2× más rápido; pero el rendimiento está «impulsado principalmente por la capacidad del orquestador y no por la de los subagentes», y escalar los subagentes rinde retornos «inconsistentes e ineficientes» [40].

5.3 Tres consecuencias

De esa asimetría se siguen tres cosas, y todas apuntan en dirección contraria a donde un esfuerzo de ingeniería tendería a invertirse por instinto.

(a) El cliente es el techo, no la red. El valor marginal del nodo $(N+1)$ -ésimo está acotado superiormente por la capacidad del orquestador para seleccionar entre lo que ya llega. Un presupuesto de ingeniería que compra nodos antes que un mejor selector del lado del cliente está gastando en el orden equivocado. Esto invierte la intuición que el marco del enjambre invita a tener y es, a mi juicio, la conclusión más accionable de este artículo.

(b) Seleccionar; no sintetizar. La selección basada en juez supera a la agregación por síntesis en 63,1 puntos porcentuales, y la síntesis al estilo Mixture-of-Agents pierde frente a la línea base simple de modelo único en 42 de 42 tareas [39]. Nótese que esto está en tensión directa con los resultados que la propia MoA reporta —65,1 % en AlpacaEval 2.0 frente al 57,5 % de GPT-4 Omni usando solo modelos abiertos [41]—, y señalo el desacuerdo en vez de escoger el lado conveniente. El diseño lo resuelve de forma conservadora: la selección es la vía por defecto, la síntesis es la excepción invocada solo ante una costura fallida, y el protocolo registra qué vía tomó cada costura para que la cuestión pueda zanjarse con datos propios.

(c) La heterogeneidad es un activo, no un defecto. Una versión anterior de este diseño trataba la diversidad de hardware y modelos voluntarios como un problema que había que homogeneizar. La evidencia apunta en sentido contrario: los equipos diversos alcanzan tasas de victoria del 81 % donde los homogéneos alcanzan el azar, y las salidas homogéneas empatan el 100 % de las veces: un selector al que se le dan candidatos idénticos no tiene nada que seleccionar [39]. Se ha reportado que los pares de modelos de familias distintas eliminan más del 30 % de los errores [42].

El zoológico de modelos de la red voluntaria es, por tanto, el sustrato que hace funcionar la selección. El protocolo debería preservar la diversidad deliberadamente: despachar los fragmentos críticos a trabajadores de familias de modelos *diferentes*, no meramente a máquinas diferentes. Es una reversión genuina del diseño anterior, y adoptarla no cuesta nada.

5.4 Dónde la tesis carece de respaldo: la hipótesis de atomicidad

La forma más fuerte de la afirmación —que un modelo de 3 B respondiendo a una subtarea atómica iguala a un modelo de frontera— no está establecida, y publicarla sin matices sería la frase más atacable del artículo.

Lo que sí está respaldado es más estrecho. Un artículo de posición de NVIDIA sostiene que los SLM son «suficientemente potentes» e «intrínsecamente más adecuados» para subtareas agénticas estrechas y repetitivas, y propone sistemas heterogéneos que invocan un modelo grande solo donde se requiere capacidad conversacional general; pero es explícitamente una pieza de discusión, no un estudio de evaluación comparativa [43]. El resultado de 8 B empatando con 32 B citado arriba [40] es real, y es un resultado *a nivel de sistema* que el mismo artículo atribuye al orquestador y no a los trabajadores.

Lo que *no* está establecido es equivalencia general alguna. Y existe un mecanismo específico por el que la afirmación puede fallar: **un trabajador más pequeño necesita más contexto para hacer el mismo trabajo.** El conocimiento que el modelo no posee en sus parámetros debe suministrarse en el prompt. Esa es la cuarta fila de la tabla de la sección 4.2, y es la razón por la que la capacidad del trabajador pertenece a la tensión del presupuesto de contexto y no a una

discusión aparte. Encoger al trabajador no es gratis; se paga en S , y S se paga en privacidad y en coste.

Por eso lo enuncio como hipótesis con un protocolo de medición, y no como afirmación:

H2 (Sustitución de capacidad). Para microtareas atómicas, bien especificadas y verificables, existe un presupuesto de contexto S al que la calidad del fragmento de un trabajador de 3-8 B es estadísticamente indistinguible de la de un modelo de frontera en la misma microtarea; y el S requerido decrece a medida que aumenta la capacidad del trabajador.

Protocolo. Fijar un conjunto de microtareas que abarque las categorías de la sección 11.1. Para cada uno de {3 B, 8 B, frontera}, barrer S y puntuar los fragmentos con juicio ciego por pares. Reportar el S al que el intervalo de confianza sobre la tasa de victoria cruza 0,5, por categoría. Reportar las categorías en las que no existe tal S en el rango barrido: esas son las categorías que Swarmby debe rechazar.

La hipótesis complementaria gobierna al cliente:

H3 (Conversión). Un selector del lado del cliente sobre $k > 1$ trabajadores heterogéneos recupera una fracción especificada de la calidad de la selección oráculo, donde el oráculo escoge el mejor fragmento disponible.

Protocolo. Con $k \in \{1, 2, 3\}$ y diversidad de familia de modelos forzada, calcular la calidad realizada frente a la selección oráculo. Reportar la fracción de recuperación y su dependencia del tamaño del propio modelo del orquestador. Una fracción de recuperación que no mejore con el tamaño del orquestador falsaría la premisa de la consecuencia (a).

Un orquestador de 8 B no es obviamente adecuado para este papel, y la literatura es desalentadora: en una tarea de planificación con estado, Llama-3.1-8B-Instruct puntuó cerca del 0-2 %, e incluso los modelos de frontera entraron en bucle en el 92-100 % de las pruebas cuando se les restringió con un validador externo [44]. Las evaluaciones de planificación más amplias apuntan en la misma dirección [45]. Los modelos pequeños son buenos *routers* —routers baratos redujeron los costes en más del 85 % en MT-Bench conservando el 95 % del rendimiento de GPT-4 [46]—, pero encaminar es clasificar, y planificar no lo es. Confundir ambas cosas es un error que este diseño evita deliberadamente: el router de la sección 8.1 es un clasificador, y al planificador de la sección 8.2 se le permite ser un modelo mayor que el router.

5.4.1 Un modelo de cobertura para el ensamblaje semántico

Con el plan como secuencia de referencia, la maquinaria de Lander-Waterman es aplicable, y la fuente de aleatoriedad resulta estar exactamente donde los supuestos del modelo se satisfacen.

Planteamiento. El plan D define un conjunto ordenado de unidades semánticas $U = \{u_1 \dots u_M\}$, fijado antes de que ocurra generación alguna. Cada paquete despachado K_i apunta a un subconjunto $S_i \subseteq U$: su unidad asignada más las unidades flanqueantes que lleve como contexto. La cobertura nominal es

$$c = (\sum_i |S_i|) / M$$

el número medio de paquetes que cubren una unidad.

Dónde reside la aleatoriedad. Esta es la diferencia sustantiva con la genómica, y es lo que hace que la transferencia sea legítima y no decorativa:

En la secuenciación de genomas, el elemento estocástico es **dónde caen los reads**. En Swarmby la colocación es determinista: la escoge el orquestador. El elemento estocástico es **qué paquetes vuelven**.

Los nodos voluntarios fallan, expiran, se desconectan y devuelven salida inservible, de forma independiente y a una tasa que la red puede medir. Sea p esa probabilidad de pérdida por paquete. La cobertura efectiva es entonces

$$c_{\text{eff}} = c \cdot (1 - p)$$

y los resultados clásicos se sostienen con c_{eff} en lugar de c :

$$\begin{aligned} P(\text{unidad } u \text{ sin cubrir}) &= e^{(-c_{\text{eff}})} \\ E[\text{unidades sin cubrir}] &= M \cdot e^{(-c_{\text{eff}})} \\ E[\text{islas de ensamblaje}] &= N_p \cdot e^{(-c_{\text{eff}} \cdot \theta)} \end{aligned}$$

donde θ es el solapamiento semántico mínimo detectable, expresado como la fracción de las unidades de un paquete que debe compartirse con un vecino para que el ensamblador las alinee: el análogo directo del parámetro de solapamiento detectable de Lander-Waterman.

La ecuación de diseño. Invertir el primer resultado da un requisito de redundancia derivado de una tolerancia declarada en vez de supuesta:

$$c \geq \ln(1/\epsilon) / (1 - p)$$

para una fracción objetivo ϵ de unidades sin cubrir. Esto sustituye el umbral arbitrario de las versiones anteriores por una tabla:

Tasa de pérdida p	$\epsilon = 5 \%$	$\epsilon = 1 \%$	$\epsilon = 0,1 \%$
0,05	$c \geq 3,2$	$c \geq 4,8$	$c \geq 7,3$
0,10	$c \geq 3,3$	$c \geq 5,1$	$c \geq 7,7$
0,20	$c \geq 3,7$	$c \geq 5,8$	$c \geq 8,6$

Como la replicación es el contribuyente dominante a c , el rango operativo práctico es de **$k = 3\text{--}5$ réplicas por unidad crítica**, con el despacho especulativo (sección 7.6) actuando como mecanismo adaptativo que eleva c_{eff} bajo demanda en vez de pagar el c del peor caso en cada petición. Esto unifica dos mecanismos que las versiones anteriores trataban como no relacionados.

Alcance, y una afirmación. El modelo acota la *disponibilidad*: la probabilidad de que una unidad semántica quede sin responder. No modela la corrección semántica —una unidad puede estar cubierta por cinco réplicas que coincidan todas y estén todas equivocadas—, razón por la cual la corrección se aborda por separado mediante el consenso y la puntuación de confianza (sección 8.4b) y mediante la verificación (sección 9.3).

Dentro de ese alcance, creo que este es **el primer modelo de cobertura publicado para el ensamblaje semántico**, y es el punto en el que el marco genómico deja de ser un vocabulario y pasa a ser una derivación. Aporta una ecuación de diseño donde el trabajo previo en este ámbito tenía una conjetura, e identifica con precisión dónde se sostiene la analogía: no en el muestreo, que Swarmbly controla, sino en la pérdida, que no controla. Los parámetros θ y la granularidad de la unidad requieren calibración empírica para el lenguaje natural; la sección 11 especifica cómo.

5.5 El enunciado honesto de la tesis del enjambre

El enjambre suministra **cobertura**; el cliente suministra **conversión**. Los nodos adicionales elevan la cobertura con retornos logarítmicos y no hacen nada por la conversión. La heterogeneidad entre nodos es lo que hace posible la selección, y debería preservarse en vez de eliminarse por ingeniería. El techo de calidad del sistema lo fija el selector del cliente, y el tamaño de trabajador que basta es una función del presupuesto de contexto y no una constante.

Esto es más débil que «un modelo de 3 B iguala a GPT-4 en tareas atómicas». También es defendible, accionable, y te dice dónde gastar.

6. Arquitectura

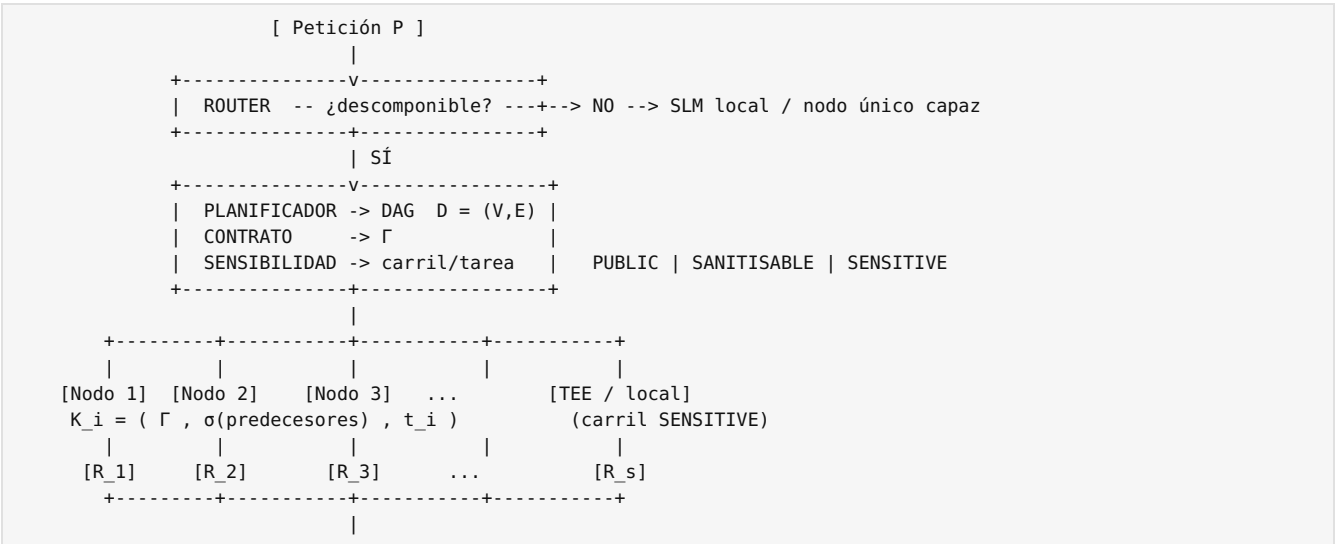
6.1 Papeles

Cliente / Orquestador — router, planificador, generador del contrato, clasificador de sensibilidad, empaquetador, despachador especulativo, verificador, ensamblador, auditor de coherencia. Requiere un SLM (se recomienda ≥ 8 B; la suficiencia de esa cifra es H3) más un modelo de embeddings.

Nodo trabajador — declara un perfil, ejecuta microtareas, emite compromisos de verificación y telemetría. Ejecuta un modelo pequeño completo.

Servicios de red — descubrimiento de pares (DHT), registro de reputación, contabilidad de créditos, muestreador de auditoría. Deliberadamente mínimos; sin blockchain en la v0.2.

6.2 Ciclo de vida de una petición



```

+-----V-----+
| VERIFICAR  compromiso LSH + auditoría muestreada
| ENSAMBLAR  seleccionar > empalmar > puente
| AUDITAR    coherencia, por costura
+-----+-----+
              v
[ Respuesta + informe de coherencia ]

```

La ejecución avanza por niveles topológicos: todas las tareas de un nivel se despachan de forma concurrente; un nivel comienza cuando sus predecesores han devuelto resultado y este ha sido verificado.

6.3 Modelo de latencia

La afirmación $\max \ll \Sigma$ es direccionalmente correcta e incompleta. El modelo honesto es

```
T_total = T_plan + Σ_levels E[ max_{i ∈ level} t_i ] + T_verify + T_assemble
```

con cuatro términos que la versión ingenua omite.

Dos de esos términos son locales y, por tanto, previsibles. La planificación escala con $|P|$ y se ejecuta en el modelo pequeño del cliente, y el ensamblaje escala con $\Sigma|R_i|$ y no con la longitud de la respuesta, lo que lo convierte en un coste que crece con el material devuelto en vez de en una constante. Los otros dos los fija el enjambre, y son donde el modelo se aparta con más nitidez de $\max \ll \Sigma$.

El primero de ellos es la cola de rezagados. $E[\max]$ sobre W extracciones concurrentes crece con W , y las colas de los voluntarios son pesadas: el ciclo de servicio efectivo medido en BOINC es de $\approx 0,61$ (0,81 conectado $\times$ 0,84 activo $\times$ 0,899 de eficiencia de CPU) y la vida mediana de un anfitrión es de 91 días [47, 48]. Los reintentos lo agravan. Con probabilidad de fallo por nodo p , $P(\text{al menos un fallo}) = 1 - (1-p)^W$; con $p = 0,10$ y $W = 20$ eso es el 88 %, de modo que casi toda petición reintenta al menos una vez y un tiempo de espera fijo de 10 segundos añadiría por tanto ≥ 10 s a la ruta crítica de forma rutinaria. El protocolo exige en su lugar **peticiones especulativas** al p95 de la distribución de latencia observada por clase (sección 7.6).

6.4 Perfiles de trabajador y determinismo

Un trabajador que ejecuta un modelo no declarado, o una cuantización distinta, altera en silencio la calidad y el registro del fragmento. El perfil forma parte, por tanto, del protocolo, y queda ligado al compromiso de verificación:

```
profile = (model_family, model_version, quantization, prompt_template_id, sampling_params, seed_policy)
```

El orquestador agrupa cada nivel del DAG en **clases de capacidad homogéneas** para controlar el desajuste de registro, preservando a la vez deliberadamente la **diversidad de familias entre las réplicas redundantes** de una misma tarea (sección 5.3(c)). Ambos objetivos están en tensión y el protocolo hace explícita la disyuntiva: homogeneidad *dentro* del papel de un fragmento, diversidad *entre* los candidatos a ese mismo fragmento.

Sobre la economía del trabajador, las GPU domésticas ociosas empleadas para inferencia de LLM se han medido en 0,111–0,149 \$ por millón de tokens en una RTX 4090, al 62–78 % del rendimiento de una H100 por aproximadamente la mitad del coste [49]. Esta es la cifra sobre la que descansa el argumento de la participación.

7. Especificación del protocolo (v0.2)

Esta sección está escrita para ser implementable. Los nombres de campo son normativos; las codificaciones se dan en JSON por claridad y MAY ser CBOR en la red.

7.1 Identificadores

- `session_id` — 128 bits aleatorios, generados por petición, nunca reutilizados.
- `task_id` — BLAKE2b-128(`session_id` || `level_index` || `task_index`), truncado a 16 bytes, codificado en hexadecimal.
- `attempt_id` — `task_id` || ':' || `attempt_counter`.

Un trabajador conoce `task_id` y `attempt_id`. MUST NOT conocer `session_id`; la derivación es unidireccional, de modo que dos trabajadores no puedan determinar que poseen fragmentos de la misma sesión comparando identificadores. Esto no anula la correlación temporal (sección 9.2), ni se afirma que lo haga.

7.2 Contrato global Γ

```

{
  "v": "0.2",
  "objective": "string – what the complete response must accomplish",
  "audience": "string",
  "register": "formal|neutral|informal|technical",

```

```

"format": "prose|markdown|json|code",
"target_len": 0,
"lexicon": { "prefer": ["..."], "forbid": ["..."] },
"entities": [ { "name": "...", "canonical": "...", "role": "..." } ],
"style_seed": "string — deterministic style anchor shared by all workers",
"budget": { "max_out_tokens": 0 }
}

```

entities es el mecanismo que impide la denominación inconsistente entre fragmentos: el análogo, en el ensamblaje, de un sistema de coordenadas compartido. style_seed es una frase corta y fija que se instruye a todos los trabajadores para que igualen en registro, lo que cuesta un puñado de tokens y reduce materialmente la deriva de registro.

[Γ] es el término dominante del presupuesto de contexto S y es por tanto el objeto de la línea de investigación en compresión identificada en la sección 4.3.

7.3 Paquete de tarea

```

{
  "v": "0.2",
  "attempt_id": "hex",
  "contract": { /*  $\Gamma$  */ },
  "predecessors": [ { "task_id": "hex", "summary": "string", "tokens": 0 } ],
  "task": {
    "instruction": "string",
    "kind": "extract|classify|generate|summarize|transform|judge",
    "expects": { "format": "...", "min_tokens": 0, "max_tokens": 0 }
  },
  "constraints": { "temperature": 0.0, "top_p": 1.0, "stop": ["..."] },
  "commitment_request": { "scheme": "lsh-activation-v1", "params": { "window": 32 } },
  "deadline_ms": 0,
  "lane": "PUBLIC|SANITISABLE",
  "tier": "GLOBAL|TRUSTED",
  "swarm_id": null
}

```

Los paquetes del carril SENSITIVE nunca se emiten hacia nodos abiertos; se ejecutan localmente o en un extremo TEE atestiguado (sección 9.4). El campo tier es ortogonal a lane: nombra la *población* que un paquete puede alcanzar, no la sensibilidad de su contenido; TRUSTED exige un swarm_id no nulo; y los paquetes clasificados para ejecución exclusivamente local no llegan siquiera a serializarse como paquetes de tarea (sección 9.5).

7.4 Resultado

```

{
  "v": "0.2",
  "attempt_id": "hex",
  "text": "string",
  "profile": { "model_family": "...", "model_version": "...", "quantization": "...",
    "prompt_template_id": "...", "sampling_params": { }, "seed": 0 },
  "commitment": { "scheme": "lsh-activation-v1", "digest": "base64", "bytes": 0 },
  "telemetry": { "gen_ms": 0, "queue_ms": 0, "tokens_out": 0, "energy_j": null },
  "sig": "ed25519 signature over the canonical serialization of all preceding fields"
}

```

energy_j es opcional y alimenta la contabilidad de sostenibilidad de la sección 10.3; es anulable porque la mayor parte del hardware doméstico no puede informarlo.

7.5 Anuncio de perfil de nodo

```

{
  "node_id": "ed25519 public key",
  "models": [ { "family": "...", "version": "...", "quantization": "...",
    "ctx": 0, "tok_per_s_est": 0 } ],
  "capabilities": { "tee": false, "attestation": null },
  "swarm": { "swarm_id": null, "registry": null, "mtls_cert_fingerprint": null },
  "resources": { "vram_mb": 0, "ram_mb": 0 },
  "policy": { "max_tokens_per_task": 0, "kinds": ["extract", "generate"] },
  "reputation": { "completed": 0, "audit_pass_rate": 0.0, "since": "ISO-8601" }
}

```

7.6 Despacho, especulación y reintento

1. Filtrar candidatos primero por nivel —un paquete marcado TRUSTED sólo se ofrece a nodos cuya clave pública figure

en la lista blanca del enjambre nombrado— y después por capacidad declarada, soporte de kind y RTT observado.

2. Para una tarea de criticidad k , despachar a k nodos seleccionados de modo que se **maximice la diversidad de familias de modelos** dentro de la clase de capacidad.
3. Arrancar un temporizador de especulación en el **p95** de la distribución de latencia observada *para ese tipo de tarea y ese presupuesto de tokens*, no en una constante fija. Al vencer, despachar una réplica adicional; aceptar el primer resultado que verifique.
4. Cancelar las réplicas pendientes al aceptar. Registrar las cancelaciones: un nodo cuyo trabajo se cancela habitualmente es lento, no deshonesto, y la reputación debe distinguir ambas cosas.
5. Ante un fallo de verificación, volver a despachar excluyendo al nodo que falló y registrar el evento para el muestreador de auditoría.

7.7 Tabla de parámetros

Parámetro	Símbolo	Valor por defecto	Derivación
Presupuesto de contexto	S	barrido	Sección 4; ningún valor por defecto es honesto antes de V_0
Tasa de redundancia	ρ	medida	Reportada, no configurada
Umbral de costura	τ_{sem}	calibrado	Sección 8.5; nunca una constante
Umbral del router	τ_{route}	calibrado, asimétrico	$\beta < 1$ en F_β , según el fundamento de Tversky de SoT-R [12]
Réplicas por criticidad	k	1 (3 para las críticas)	Guiado por coste; la mayoría de k sigue la práctica de BOINC [47]
Disparo de especulación	—	p95 por clase	Sección 6.3
Tasa de muestreo de auditoría	λ	0,01-0,05	Sección 9.3; ajustada contra el calendario de penalizaciones [50]
Anchura máxima del plan	—	8	La cola de rezagados crece con la anchura (sección 6.3)
Profundidad máxima del plan	—	4	Más allá de esto, la densidad de dependencias aconseja no fragmentar

8. Algoritmos

8.1 Router

```
function is_decomposable(P) -> (bool, score)
  f ← features(P):
    · señales de tipo de tarea (extraer / enumerar / resumir vs demostrar / derivar / refactorizar)
    · longitud de P
    · densidad de marcadores de dependencia secuencial ("luego", "usando el resultado", "paso N")
    · presencia de estado mutable compartido (código, libros contables, totales acumulados)
    · petición de un único artefacto vs de un conjunto de elementos
  score ← classifier(f) # un modelo de clase 120M es suficiente [12]
  return (score >  $\tau_{\text{route}}$ , score)
```

τ_{route} se calibra con **F_β , $\beta < 1$** : un falso positivo (fragmentar algo que no debería fragmentarse) cuesta más que un falso negativo. Esta es la lección de SoT-R, y es la diferencia entre un sistema que mejora lo de 2023 y uno que retrocede respecto de ello.

8.2 Planificador

```
function plan(P) -> D = (V, E)
  units ← identify_semantic_units(P)
  for each ordered pair (u, v):
    E ← E ∪ {(u,v)} si v requiere el *resultado* de u, no meramente su enunciado
  assert acyclic(D)
  if width(D) == 1: return REFUSE # una cadena no es paralelizable; no fingir lo contrario
  if depth(D) > max_depth: return REFUSE
  return D
```

La distinción entre requerir el *resultado* de un predecesor y requerir su *enunciado* es el meollo. El prompting de menos a más alcanza $\geq 99\%$ en SCAN frente al 16% de la cadena de pensamiento precisamente por respetar secuencialmente las dependencias de resultado [51]; un planificador que confunde una dependencia de resultado con una de enunciado convierte esa ganancia en pérdida.

8.3 Empaquetado

```
function build_packet( $\Gamma$ ,  $t_i$ , preds,  $S_{\text{target}}$ ) ->  $K_i$ 
  base ←  $\Gamma$  # nunca se elide
  budget ←  $S_{\text{target}} - |\Gamma|$ 
  ordenar preds por (peso de arista, recencia)
  for p in preds while budget > 0:
    s ← summarize(p.result, min(budget, cap_per_pred))
```

```
attach(s); budget -= |s|
return ( $\Gamma$ , attached,  $t_i$ )
```

Γ nunca se recorta para ajustarse a un presupuesto. Si $S_{\text{target}} < |\Gamma|$, el planificador reduce N en su lugar: pocos fragmentos grandes ganan a muchos pequeños, según el resultado de LongRAG, donde unidades de recuperación de 4 K tokens y menos de ocho unidades principales igualaron al estado del arte plenamente entrenado sin entrenamiento alguno [52].

8.4 Ensamblaje

```
function assemble(fragments, D,  $\Gamma$ ,  $\tau_{\text{sem}}$ ) -> (text, seam_report)
  ordered  $\leftarrow$  topological_flatten(D)
  chosen  $\leftarrow$  []
  for t in ordered:
    cands  $\leftarrow$  fragments[t]
    chosen.append( cands[0] if |cands| == 1
                  else judge_select(cands,  $\Gamma$ ) )      # seleccionar, no sintetizar
  out, seams  $\leftarrow$  [], []
  for (a, b) in consecutive(chosen):
    sim  $\leftarrow$  cos( embed(tail_window(a)), embed(head_window(b)) )
    if sim  $\geq \tau_{\text{sem}}$ :
      out.append(a); seams.append((a,b,"splice",sim))
    else:
      bridge  $\leftarrow$  slm.write_transition(tail(a), head(b),  $\Gamma$ )  # vía de excepción
      out.append(a); out.append(bridge)
      seams.append((a,b,"bridge",sim))
  return join(out), seams
```

Cada costura se registra con su similitud y la vía tomada. El informe de costuras forma parte de la respuesta, según P6.

8.4b Consenso por alineamiento múltiple de réplicas (E16)

La sección 8.4 resuelve fragmentos *distintos* que ocupan posiciones *distintas*. Esta sección resuelve k réplicas de la misma microtarea, es donde la analogía genómica paga su dividendo más literal, y produce la única capacidad de esta arquitectura que un proveedor centralizado no puede replicar a ningún precio.

Dos niveles, deliberadamente distintos. Swarmby ensambla en dos niveles, y confundirlos es el error contra el que advierte la sección 2.4:

Nivel	Unidad	Mecanismo	Cuándo
Macro	Subtareas distintas de una misma tarea grande	Solapar y empalmar con contexto flanqueante (sección 8.4)	Trabajo generativo largo: informes, documentos multisección, corpus
Micro	k réplicas completas de la misma microtarea	Alineamiento múltiple y consenso (esta sección)	Toda microtarea de criticidad $k > 1$, incluida una petición que fue atómica desde el principio

Una petición atómica —una que el router se niega a descomponer— salta por completo el nivel macro y va directa al nivel micro con k réplicas. **Dividir una pregunta atómica en preguntas parciales no es una operación soportada**, porque elimina información antes del muestreo en vez de muestrear de forma redundante; ninguna cantidad de cobertura la recupera.

Algoritmo.

```
function consensus(replicas,  $\Gamma$ , U) -> (text, confidence[])
  # replicas: k respuestas completas a la MISMA microtarea,
  #           de nodos de familias de modelo deliberadamente distintas (sección 7.6)
  for r in replicas:
    r.units  $\leftarrow$  segment_into_semantic_units(r, granularity=U.granularity)

  aligned  $\leftarrow$  align_multiple(replicas.units)      # alineamiento progresivo sobre
                                                    # similitud de embeddings, con huecos

  out, conf  $\leftarrow$  [], []
  for column in aligned:
    agree  $\leftarrow$  agreement_score(column)           # fracción de réplicas cuya unidad
                                                    # es mutuamente consistente

    if agree  $\geq \alpha_{\text{high}}$ :
      out.append(majority_unit(column)); conf.append(("HIGH", agree))
    elif agree  $\geq \alpha_{\text{low}}$ :
      out.append(judge_select(column,  $\Gamma$ )); conf.append(("MEDIUM", agree))
    else:
      out.append(judge_select(column,  $\Gamma$ )); conf.append(("LOW", agree))
      flag_low_confidence_region(column)           # expuesta al usuario
  return join(out), conf
```

Por qué esto importa más allá del ensamblaje. El acuerdo entre *réplicas muestreadas de forma independiente procedentes de familias de modelo distintas* es una señal medible sobre la fiabilidad de una afirmación. Una unidad en la que convergen cinco modelos no emparentados no es por ello verdadera; una unidad en la que divergen merece señalarse de forma fiable. El protocolo devuelve, por tanto, junto a la respuesta, un **mapa de regiones de baja confianza**, directamente análogo a las puntuaciones de calidad por base que un ensamblador de genomas reporta en lugar de emitir una secuencia uniformemente confiada.

Ningún sistema comercial ampliamente desplegado devuelve esto hoy, y la razón es arquitectónica antes que comercial: **un solo modelo no tiene nada contra lo que alinearse a sí mismo**. Muestrear repetidamente un modelo mide su propia varianza, no el desacuerdo entre estimadores independientes. La señal existe aquí porque la red es heterogénea y distribuida, lo que significa que la redundancia que la descentralización *exige* es la misma redundancia que produce el mapa de fiabilidad. Un coste de la arquitectura y una capacidad de ella resultan ser el mismo mecanismo visto desde dos direcciones.

Para un usuario, esta es la diferencia entre una respuesta y una respuesta que le dice cuáles de sus partes conviene comprobar. Para un despliegue regulado, es una superficie de auditoría que la inferencia monolítica no ofrece.

Tres salvedades honestas.

1. **El acuerdo no es verdad.** Los modelos entrenados sobre corpus solapados comparten errores. La convergencia sobre una falsedad común es un fallo correlacionado que el alineamiento no puede ver. Precisamente por eso la sección 7.6 exige **diversidad entre familias** entre las réplicas: la señal solo es tan fuerte como la independencia de las muestras.
2. **Debe validarse, no suponerse, y el primer intento de validarlo falló.** La correlación entre la puntuación de acuerdo y la corrección factual es una magnitud empírica. Medida contra un juez de clase par sobre 597 unidades semánticas, salió en $r = -0,030$ con tramos planos y no monótonos (sección 11.3). Esa medición es débil en sus propios términos —el juez aceptó el 93,3 % de las unidades, dejando casi ninguna varianza contra la que pudiera aparecer una correlación—, de modo que deja el mecanismo **sin sustento y no refutado**. La sección 11.4 especifica el experimento contra conjuntos de datos con verdad de referencia que zanjaría la cuestión. Hasta que se ejecute, las etiquetas de confianza se reportan como *acuerdo*, nunca como *exactitud*, y el mapa no se ofrece como garantía de fiabilidad.
3. **Cuesta $k\times$.** El consenso se aplica por criticidad, no universalmente.

8.5 Calibración de umbrales

```
function calibrate_tau(labelled_pairs, embedder,  $\beta = 0.5$ ) -> ( $\tau^*$ , curve)
  sims  $\leftarrow$  [ cos(embed(a.tail), embed(b.head)) for (a,b,label) in pairs ]
  for  $\tau$  in quantiles(sims, 200):
    calcular precisión/exhaustividad de "es una costura rota"
     $F_\beta \leftarrow (1+\beta^2) \cdot P \cdot R / (\beta^2 \cdot P + R)$ 
  return argmax_ $\tau$   $F_\beta$ , curve
```

$\beta < 1$ pondera la precisión: declarar rota una costura dispara una reescritura, y las reescrituras innecesarias son la manera en que un sistema degrada texto que ya estaba bien. τ debe rederivarse cada vez que cambia el modelo de embeddings, por las razones de anisotropía de la sección 4.4.

8.6 Auditoría de coherencia

Dos instrumentos, reportados por separado y nunca fundidos en una única puntuación de «calidad» —fundirlos es exactamente el modo en que se esconde el daño [12]:

1. **Coherencia local por rejilla de entidades** [53] — menciones de entidades y papeles gramaticales a lo largo de las oraciones, puntuados a partir de probabilidades de transición. La línea base estándar para detectar el daño por reordenación e inserción.
2. **Taxonomía de errores de costura** — el subconjunto mecánicamente detectable de las clases de error identificadas a partir de 1.193 anotaciones humanas sobre 100 libros [54]: omisión de entidad, contenido duplicado, contradicción, cambio de registro o de tiempo verbal, referencia colgante, transición ausente, reintroducción repetida y denominación inconsistente. Se reportan como recuentos y como fracción de oraciones libres de cualquier error detectado.

La cifra destacada es el **impuesto de coherencia**: la degradación relativa frente a la generación monolítica con el mismo modelo.

9. Privacidad, verificación y nodos adversarios

9.1 La fragmentación no es cifrado

El desarrollo previo de este concepto describía la fragmentación descontextualizada como una forma de «cuasi-cifrado»,

con el razonamiento de que un nodo que posee un fragmento sin contexto global no posee nada de valor. Ese razonamiento no sobrevive al contacto con la literatura de reidentificación, y la afirmación se retira.

Cuatro hallazgos de esa literatura inciden en el razonamiento, y todos apuntan en la misma dirección. El primero es que los cuasi-identificadores bastan: la combinación de código postal, fecha de nacimiento y sexo identifica de forma única a aproximadamente el 87 % de la población de EE. UU. pese a que ninguno de los tres es un identificador por sí solo [55] y, aunque una revisión posterior sitúa la cifra cerca del 63 %, eso no resulta tranquilizador. Toda defensa por fragmentación o generalización sintáctica propuesta en esa literatura ha sido rota después por un ataque [56], y los datos dispersos de alta dimensión resultan ser intrínsecamente reidentificables a partir de un puñado de atributos gruesos y ruidosos [57].

El segundo es que el estilo es en sí mismo un identificador. La atribución de autoría opera a escala de internet [58], sobrevive al acortamiento y al desplazamiento de dominio entre plataformas [59], y funciona por debajo de los 280 caracteres [60]. Un fragmento «sin contexto» sigue portando, por tanto, la firma estilística de quien lo solicita y, aunque en el caso de Swarmbly el fragmento lo genera un *trabajador*, el microprompt derivado del texto del usuario no. El tercero es que las representaciones intermedias se invierten: los embeddings de texto revelan casi tanto como el propio texto [61] y los prompts pueden recuperarse solo a partir de las salidas del modelo [62].

El cuarto es decisivo, porque no procede por analogía sino que ataca esta misma arquitectura. En la inferencia partida — un cliente que computa parte de una red y un servidor el resto—, el ataque ActInv alcanza precisión y exhaustividad por encima del 98 % en casi todos los casos evaluados, con ROUGE-L consistentemente por encima de 0,96. Cortar tras dos bloques de cliente de Qwen3-0.6B produce un 99,76 % de precisión, e incluso con siete bloques retiene el 77,74 %. Las defensas rinden por debajo de lo esperado: con un 70 % de esparcimiento de activaciones la precisión decrece «solo modestamente», y hace falta ruido gaussiano con varianza 10^{-1} antes de que la recuperación se degrade sustancialmente [63].

Swarmbly no transmite activaciones, lo que lo sitúa en mejor posición que la inferencia partida. Pero la dirección de la evidencia es inequívoca, y hay un argumento adicional interno a este mismo diseño: **la sección 4.2 establece que la coherencia exige enviar el contrato global Γ a cada trabajador**. Un nodo que posee Γ posee el objetivo, la audiencia, el formato y las restricciones de la sesión. Descontextualización y coherencia son antagonistas por construcción, y ninguna cantidad de ingeniería disuelve eso.

9.2 Lo que sí puede afirmarse

Swarmbly reduce la superficie de exposición respecto de un proveedor centralizado que lee y retiene el prompt completo, y respecto de los esquemas de paralelismo de tubería en los que los nodos observan activaciones intermedias y el texto en curso de generación. No proporciona **ninguna garantía criptográfica de confidencialidad**. Un adversario que controle una fracción significativa de los nodos, o que correlacione por temporización e identificador de sesión, puede reconstruir una porción sustancial de una sesión.

Eso es defendible, sigue siendo una mejora que vale la pena tener, y puede decirse en una interfaz de usuario sin sonrojarse.

Dos canales residuales merecen nombrarse porque son fáciles de pasar por alto: la **correlación temporal** (los fragmentos de una sesión llegan en ráfaga; la derivación unidireccional de `task_id` de la sección 7.1 no lo oculta) y la **huella digital del contrato** (un Γ distintivo es en sí mismo un identificador de sesión entre los nodos que lo reciben). Las mitigaciones —despacho con jitter, paráfrasis del contrato por nodo— cuestan latencia y coherencia respectivamente, lo cual remite de nuevo a la sección 4.2.

9.3 Verificación

La confidencialidad criptográfica fuerte es inasequible aquí, y vale la pena enunciar las cifras para que la conclusión no se confunda con derrotismo. La computación multiparte de propósito general sobre un transformer se ejecuta con una ralentización del orden de 10^4 – $10^6\times$, con 280,99 GB de comunicación para una sola inferencia de BERT-Base [64, 65]; los mejores sistemas de dos partes reportan aproximadamente 8 minutos por token para LLaMA-7B [66]. Las pruebas de conocimiento cero sobre inferencia necesitan menos de 15 minutos para demostrar un paso hacia adelante de un modelo de 13 B [67]. Nada de esto cabe en una economía de voluntarios.

Lo que sí cabe es un esquema de dos capas:

Capa 1 — integridad computacional por compromiso sensible a la localidad. Un esquema de compromiso sobre las activaciones detecta la sustitución no autorizada de modelo, prompt o precisión con 100 % de exactitud, cero falsos positivos y cero falsos negativos en las pruebas reportadas, a 258 bytes por cada 32 tokens —aproximadamente $1000\times$ de compresión frente a los embeddings en bruto—, validando más rápido que la inferencia original y manteniéndose robusto entre tipos de GPU y reordenaciones del cómputo [68]. Esto es lo que hace posible un mercado de nodos: cuesta casi nada y cierra el fraude obvio, que es un nodo que anuncia un modelo de 8 B y sirve uno de 1 B.

Capa 2 — auditoría pública muestreada. Verificación a aproximadamente el 1 % del coste de la inferencia, segura bajo un supuesto de *un verificador honesto* en vez de mayoría honesta, con probabilidad de fallo $P_{\text{fail}} \sim \rho^k$ para una tasa de corrupción ρ y un comité de tamaño k . La propiedad de diseño esencial: **los trabajadores no pueden**

distinguir una tarea de auditoría de una real [69]. La tasa de auditoría λ y el calendario de penalizaciones se fijan conjuntamente; la relación entre tasa de muestreo, magnitud de la penalización y equilibrio de juego honesto se formaliza en [50].

Capa 3 — la selección como defensa. Con $k > 1$ réplicas, la selección basada en juez de la sección 8.4 ya descarta los fragmentos anómalos como efecto colateral de mejorar la calidad [39]. Es la defensa más barata del sistema porque la paga otra cosa.

Lo que ninguna de estas capas hace es verificar la *fidelidad semántica*. La capa 1 demuestra que un modelo declarado se ejecutó sobre una entrada declarada; no demuestra que la prosa resultante sea verdadera, ni inocua. Esa brecha importa porque cada fragmento devuelto es entrada no confiable que fluye hacia el modelo del cliente, terreno directo de la inyección de prompts, la cadena de suministro, el manejo indebido de salidas y las debilidades de embeddings en la guía actual de seguridad de aplicaciones [70]. Y no cabe confiar en que el cliente lo advierta: los modelos de razonamiento estándar atribuyen los fallos en sistemas agénticos con menos del 10 % de exactitud [71]. Un sistema que no puede atribuir fallos *honestos* no detectará los adversarios. La respuesta del protocolo es acotar el radio de daño: los fragmentos son datos, nunca instrucciones; el ensamblador se ejecuta con defensas de manejo de salida; y se imponen esquemas de salida específicos por kind antes de que un fragmento entre en el contexto de ensamblaje.

9.4 Carriles de sensibilidad

Carril	Criterio	Destino	Coste
PUBLIC	Sin PII, sin secreto comercial	Nodos voluntarios abiertos	Ninguno
SANITISABLE	PII detectable y seudonimizable	Nodos abiertos; rehidratado localmente	Riesgo residual real (abajo)
SENSITIVE	Salud, jurídico, financiero, identificable	Ejecución local, o TEE atestiguado	<7 % de sobrecoste medio en computación confidencial sobre H100, por debajo del 5 % para consultas típicas y tendiendo a cero a medida que crece el tamaño del modelo [72]; mediciones independientes bajo Intel TDX reportan 8,9–21,8 % según el régimen [73]

El carril TEE es lo que hace el protocolo adoptable por una organización, y es asequible: un sobrecoste porcentual de un solo dígito es la única primitiva de confidencialidad en este espacio con esa propiedad.

El carril SANITISABLE debe describirse a los usuarios con honestidad. Frente a un modelo sin defensas sobre un corpus de textos jurídicos, la extracción de PII alcanza ~23 % de exhaustividad y ~30 % de precisión, y la *inferencia* de PII a partir de 100 candidatos alcanza el 70 %, 50 % y 28 % en tres corpus. La privacidad diferencial con $\epsilon=8$ reduce la exhaustividad de extracción a alrededor del 3 %, pero no a cero [74], y la generación con privacidad diferencial degradada de forma medible la calidad del lenguaje [75]. La sanitización reduce el riesgo; no lo elimina, y la interfaz debería decirlo en vez de enterrarlo.

9.5 Niveles dinámicos de privacidad y enjambres de confianza (topologías federadas)

Los carriles de la sección 9.4 clasifican el *trabajo*. No dicen nada sobre la *población de máquinas* que ese trabajo puede alcanzar, y es el supuesto tácito de una única bolsa indiferenciada de voluntarios anónimos lo que obliga al carril SENSITIVE a la elección estrecha entre ejecución local y hardware atestiguado. Hay un segundo eje disponible, y añadirlo cuesta casi nada: la propia topología puede escalonarse, de modo que la decisión de enrutamiento sea un par — qué carril, y qué malla.

La clasificación, y dónde se ejecuta. Toda petición pasa por un clasificador de privacidad antes de la planificación, y ese clasificador opera en dos modos. El primero es una **bandera manual dura** `---privacy=trusted`, `--privacy=local`— que es determinista, la declara el usuario y nunca queda anulada por la vía automática; quien afirma que un documento es confidencial no está pidiendo una segunda opinión. El segundo es un **triaje automático**: un modelo pequeño local realiza reconocimiento de entidades nombradas sobre el prompt y eleva el nivel cuando detecta entidades de clases reguladas, entre ellas identificadores personales, datos de salud y financieros, credenciales y nombres de proyectos internos.

La propiedad esencial es que ese clasificador se ejecuta **íntegramente en el cliente**. Un clasificador de privacidad que consulta a la red para decidir si el prompt es privado ya ha revelado el prompt, y no existe configuración bajo la cual eso sea aceptable; el modelo de triaje forma parte, por tanto, de la pila del cliente y no es un servicio. Está además deliberadamente orientado a la exhaustividad —debe sobreclasificar—, porque el coste de encaminar un prompt público hacia un enjambre de confianza es algo de rendimiento, mientras que el coste del error inverso es exactamente el fallo que el nivel existe para evitar. La honestidad de la sección 9.4 sobre la sanitización se aplica aquí sin cambios: el triaje automático *eleva* un nivel, no *certifica* la ausencia de contenido sensible, y la interfaz debe decirlo.

Los tres niveles. El **nivel 1, la malla global no confiable**, es el predeterminado y es la red que este documento ha descrito hasta aquí: nodos voluntarios abiertos, carriles PUBLIC y SANITISABLE, la pila de verificación completa de la sección 9.3 y redundancia en el k derivado en la sección 5.4.1.

El **nivel 2, el enjambre de confianza**, es una submallá permissionada. La pertenencia es una lista blanca criptográfica de claves públicas de nodo bajo un registro controlado por el operador; cada enlace lleva TLS mutuo, de modo que ambos extremos quedan autenticados y no sólo el cliente; y el despliegue típico es una LAN corporativa, una red de campus o una superposición VPN. El protocolo no cambia —un enjambre de confianza es el mismo protocolo sobre una población restringida, no un segundo protocolo— y esa restricción es deliberada. Los tiempos de ida y vuelta dentro de un enjambre así se desploman de decenas o centenas de milisegundos a bastante menos de uno, lo que significa que la asimetría de ancho de banda de la sección 3, la razón entera para fragmentar el problema en lugar del modelo, queda localmente suspendida. Sería técnicamente posible ejecutar una partición de grano más fino dentro del cortafuegos. Swarmblly no lo hace, porque un despliegue que se comporta de una manera dentro del perímetro y de otra fuera son dos sistemas que implementar, verificar y razonar. Lo que compra la baja latencia es, en cambio, holgura en el presupuesto de contexto S : contextos por fragmento K_i mayores, solapamiento más generoso, una razón de redundancia ρ más alta y, por tanto, mejor coherencia con el mismo tiempo de reloj — una mejora obtenida gastando el mismo parámetro de diseño y no introduciendo un mecanismo nuevo.

El **nivel 3, ejecución puramente local**, se selecciona con `--privacy=local` y significa lo que dice: ningún paquete abandona la máquina. La planificación, la generación y el ensamblaje los realiza el modelo del cliente, la capacidad queda acotada por el hardware local y la degradación frente al nivel 1 es el precio honesto de la garantía. Es el único nivel en el que Swarmblly formula una afirmación incondicional de confidencialidad, y puede formularla precisamente porque no hay red sobre la cual formularla.

La redundancia en un enjambre de confianza, y qué cuesta reducirla. El parámetro k ha servido a dos propósitos simultáneos a lo largo de todo el documento, y el escenario del enjambre de confianza es el primer contexto que los separa. La **redundancia adversarial** defiende frente a un nodo que miente sobre su trabajo; es el supuesto detrás de la sección 9.3 y de la discusión de Sybil que sigue, y es exactamente lo que elimina una lista blanca criptográfica, puesto que la defensa por voto mayoritario contra la generación deshonesta dentro de una población de máquinas autenticadas y contractualmente responsables está pagando por una amenaza ya eliminada en la capa de identidad. La **redundancia epistémica** es otra cosa: el mapa de confianza de la sección 8.4b necesita k réplicas generadas de forma independiente por familias de modelos distintas para tener algo que alinear, y no es una defensa frente a nadie. Reducir k a 1 elimina la segunda junto con la primera.

El protocolo permite, por tanto, la reducción, pero no permite que sea silenciosa. Un enjambre de confianza **MAY** fijar la criticidad en $k = 1$ por coste o latencia, lo cual es una decisión de operación legítima; cuando lo hace, los metadatos de la respuesta registran que no se produjo mapa de confianza y el cliente expone esa ausencia de forma explícita en lugar de presentar un mapa vacío como si fuera acuerdo. Un operador que elige $k = 1$ está eligiendo rendimiento por encima de una superficie de auditoría, y el papel de la especificación es hacer visible esa elección, no tomarla. La cobertura acota el suelo de forma independiente: con $c_{\text{eff}} = c(1 - \rho)$ y $c = 1$ no hay margen alguno frente a la pérdida, de modo que la ecuación de diseño de la sección 5.4.1 exige $k \geq 2$ siempre que la tasa de pérdida intra-enjambre medida p supere la tolerancia ϵ , por fiable que sea la LAN.

Por qué el nivel importa más allá de la ingeniería. Los regímenes de protección de datos están escritos alrededor de encargados de tratamiento identificables y vinculados contractualmente: las relaciones de encargado del RGPD, los acuerdos de socio comercial bajo HIPAA y sus equivalentes en otras jurisdicciones presuponen todos una entidad que pueda nombrarse, auditarse y responsabilizarse. Un voluntario anónimo no puede ser encargado bajo ninguno de ellos. Un nodo en lista blanca, mutuamente autenticado y dentro del registro del propio operador, sí puede. El nivel 2 no es, por tanto, una prestación de rendimiento; es la construcción bajo la cual esta arquitectura resulta lícita en entornos donde el nivel 1 no lo es, y convierte «no puede usarse con datos de pacientes» en «puede usarse con datos de pacientes, en las máquinas de la propia institución, bajo el registro de la propia institución, con el mismo cliente y el mismo protocolo».

Y qué es lo que no hace. Un enjambre de confianza reubica la confianza; no la elimina. Quien controla la lista blanca controla el enjambre, lo que convierte la gobernanza del registro en una función crítica de seguridad y no en una tarea administrativa, y un miembro comprometido dentro del perímetro es *más* peligroso que un nodo no confiable fuera de él, precisamente porque la redundancia que lo habría detectado puede haberse reducido. El TLS mutuo autentica el canal y la identidad; no dice absolutamente nada sobre si el modelo detrás de esa identidad es el declarado. Por esa razón el compromiso sensible a la localidad de la sección 9.3 sigue siendo **REQUIRED** dentro de un enjambre de confianza incluso allí donde se relajan el muestreo de auditoría y el voto mayoritario. Una topología federada es un cambio de modelo de amenaza, no la ausencia de uno.

9.6 Resistencia a Sybil: una limitación, declarada

Sin una autoridad de identidad confiable, un único adversario puede presentar arbitrariamente muchas identidades distintas, derrotando **cualquier** esquema basado en redundancia o voto mayoritario [76]. Los sistemas de reputación no escapan a esto: el algoritmo P2P canónico requiere un conjunto de pares *previamente confiables* para ser resistente a Sybil, lo que reintroduce el anclaje que pretendía eliminar [77].

Que esto no sea meramente teórico se ve en el despliegue insignia de la computación voluntaria, donde el 41,4 % de los anfitriones pertenecía a usuarios de un solo anfitrión, el 44,2 % a usuarios con 2-10 anfitriones, y el mayor usuario individual operaba 2.987 anfitriones [47]: concentración extrema, por parte de un participante benigno y sin incentivo

alguno para ocultarlo.

Swarmbly no es, por tanto, resistente a Sybil en sentido fuerte, y el protocolo lo dice. Adopta una confianza por capas: reputación acumulada, un coste de entrada al registro, auditoría muestreada con penalización económica y un conjunto de nodos ancla operados por la fundación para el arranque en frío. Sistemas comparables toman la misma medicina bajo otros nombres [6].

10. Economía, gobernanza y sostenibilidad

10.1 Créditos, no tokens

El desarrollo previo proponía un preminado del 15 % para los fundadores y una comisión de protocolo del 0,5 % sobre cada micropago. Ambos se retiran por motivos regulatorios y narrativos. El marco suizo clasifica los tokens como de pago, de utilidad o de activo, con una prueba de dos condiciones para escapar de la clasificación como valor [78]; un instrumento preminado y transferible con expectativa de revalorización es el arquetipo que la activa. Las exenciones europeas son estrechas: 1.000.000 € en doce meses, o 150 personas por Estado miembro, con las normas para proveedores de servicios en vigor desde el 30 de diciembre de 2024 [79].

El diseño que queda fuera de ambos regímenes es deliberadamente poco emocionante: créditos no transferibles entre cuentas, ganados por procesar y gastados por solicitar; sin preventa y sin preminado; utilidad inmediata desde el primer día en vez de una promesa; saldos que expiran para desincentivar el acaparamiento; y conversión a moneda fiduciaria en una sola dirección: las empresas compran capacidad a través del brazo comercial, los voluntarios no venden créditos. Esto es menos emocionante que un token y es lo que permite lanzar sin asesoría en materia de valores en tres jurisdicciones.

10.2 Licencia y gobernanza

La implementación del protocolo es **AGPL-3.0-or-later**. Su cláusula 13 cierra la brecha del uso en red que la GPL deja abierta [80]. Tres matizaciones pertenecen al registro: la obligación se adhiere al Programa y a sus modificaciones, no a una pila propietaria circundante; la licencia cubre el software, no el protocolo, de modo que una reimplementación de sala limpia es lícita; y su fuerza práctica es la disuasión y no el litigio, dado que la política publicada de al menos un proveedor importante prohíbe de plano el código AGPL.

La guía empírica más sólida disponible sobre esta elección procede de la reciente oleada de recambios de licencia: cuatro de cuatro proyectos que endurecieron sus licencias produjeron una bifurcación independiente exitosa, y dos de los cuatro revirtieron después a AGPL [81, 82]. Empezar en AGPL y quedarse ahí es la posición que la historia respalda.

De ello se siguen dos decisiones estructurales. Se rechaza la licencia dual: requiere una entidad capaz de vender excepciones propietarias, lo que es incompatible con una fundación cuyo mandato es la apertura; los ingresos provienen en cambio del servicio gestionado. Y **la marca, no el copyright, es la palanca de control operativa**, siguiendo el modelo de una fundación que gobierna dos marcas denominativas y dos logotipos con una separación documentada entre lo permitido y lo sujeto a aprobación. La contribución se hace mediante firma DCO, no CLA, porque la cesión de copyright genera fricción precisamente con la comunidad que este proyecto necesita.

Sobre la estructura: una *Verein* suiza puede constituirse con rapidez y sin capital mínimo, lo que basta para ostentar derechos y recibir subvenciones; una *Stiftung* es el instrumento adecuado más adelante, cuando haya un tesoro que administrar. La afirmación de que una fundación es «inadquirible e insilenciable» es en cualquier caso exagerada: las fundaciones se capturan a través de los consejos, la dependencia de donantes y el control de los repositorios y las marcas. La independencia es una práctica, no una forma jurídica.

10.3 Sostenibilidad, no reivindicada

Los centros de datos consumieron 415 TWh en 2024, alrededor del 1,5 % de la electricidad mundial, con proyecciones de 945 TWh para 2030 [83]. Las instalaciones hiperescala estadounidenses se abastecen de redes medidas en 545 gCO₂/kWh frente a una media nacional de 370 g: un 48 % más sucias [84]. Esas cifras respaldan la *motivación*.

No respaldan una afirmación de beneficio neto, y no la hago. La energía por token varía en casi tres órdenes de magnitud entre configuraciones, y los aceleradores de centro de datos logran la menor energía por token en la gran mayoría de escenarios; el consumo en reposo de 12-90 W lo paga íntegro un nodo que está disponible pero sin uso [85]. El PUE global es de 1,54, pero los hiperescalares operan a 1,09-1,15 frente a un ~1,0 efectivo en un hogar: un margen del 9-15 %, no un orden de magnitud [86]. La redundancia añadida a eso es sobrecoste puro, y el calor residual se recupera en los centros de datos y esencialmente nunca en los hogares.

El compromiso que sí adquiero es procedimental: adoptar el estándar de Intensidad de Carbono del Software —SCI = ((E × I) + M) / R, ISO/IEC 21031:2024, que excluye explícitamente las compensaciones [87]—, instrumentar los nodos y el cliente, y **publicar el resultado sea cual sea**. El argumento motivador defendible es el del carbono incorporado: prolongar la vida útil de hardware que ya existe evita nueva fabricación, y la fabricación es la mayor parte de la huella de un gran operador, y creciente. Ese argumento merece medición, no aserción.

Sobre la demanda inducida. Abaratar un recurso normalmente aumenta su consumo total en vez de desplazar el uso

existente —la paradoja de Jevons—, y un revisor de un fondo climático lo planteará. Lo abordo de frente en vez de esperar que no se mencione.

Democratizar la inferencia inducirá muy probablemente una demanda que hoy no existe. Lo que Swarmbly afirma no es que esa demanda desaparezca, sino que **la absorbe hardware que ya ha sido fabricado**. En el modelo centralizado, la demanda inducida se atiende construyendo más centros de datos y comprando más aceleradores, que es precisamente el término de carbono incorporado que domina y que está creciendo. En Swarmbly, la demanda inducida se atiende elevando la utilización de dispositivos que ya están en el mundo, de modo que **el carbono incorporado marginal de atenderla tiende a cero**.

Dos condiciones acotan este argumento y ambas se declaran en vez de suponerse. Solo se sostiene mientras **exista capacidad ociosa**: en la saturación del enjambre, el crecimiento volvería a impulsar nueva fabricación. Y solo se sostiene para la fracción del tráfico atendida por hardware voluntario genuinamente preexistente, lo que **excluye explícitamente los nodos ancla operados por la fundación** del periodo de arranque (sección 10.4). El panel público reporta esa fracción, de modo que la afirmación pueda auditarse en vez de aseverarse.

10.4 Arranque: nodos ancla, declarados

Una red cuya oferta y demanda deben llegar simultáneamente no arranca sola. El arranque de Swarmbly subvenciona la oferta: la fundación opera capacidad alquilada para que el servicio sea rápido y estable desde el primer día, y la retira a medida que crece la oferta comunitaria.

Esto crea una exposición de integridad que el protocolo maneja mediante divulgación y no mediante omisión. Durante ese periodo una porción del «enjambre voluntario» es hardware alquilado de centro de datos, y para esa porción **el argumento del carbono incorporado no aplica y la afirmación de descentralización solo es parcialmente cierta**.

Los compromisos son, por tanto, explícitos:

- Tales nodos se denominan **nodos ancla operados por la Fundación** y se etiquetan como tales en el registro.
- El panel público reporta, en tiempo real, **la proporción del tráfico atendida por nodos ancla frente a nodos comunitarios**.
- La Fundación publica una trayectoria objetivo para esa proporción y reporta su cumplimiento.

La reducción de la proporción de anclas se convierte en una medida pública y auditable del progreso hacia la descentralización real, y en una métrica legible para un financiador. Una red que oculta su subvención tiene una bomba reputacional con temporizador; una que la publica tiene un instrumento de rendición de cuentas.

11. Evaluación

11.1 Categorías de tarea

La adecuación exige cinco atributos simultáneos: descomponible en subtareas genuinamente independientes; tolerante a la latencia; intensiva en tokens; dependencias interfragmento débiles; contenido verificable o no sensible.

Categoría	Adecuación	Razonamiento
Procesamiento documental masivo	Alta	Vergonzosamente paralelo, sin dependencias, tolerante a la latencia: el perfil que hizo funcionar la computación voluntaria en primer lugar
Generación de datos sintéticos, etiquetado	Alta	Independiente por muestra; verificable por filtrado; gran volumen
Barridos de migración de código	Alta	Independiente por archivo; verificable compilando y ejecutando pruebas
Evaluación y juicio a escala	Alta	Independiente; la agregación es un voto, la fusión más segura disponible
RAG sobre corpus grandes	Moderada	La etapa de mapeo es paralela; pero pocas unidades grandes ganan a muchas pequeñas [52]
Informes estructurados largos	Moderada	Descomponible por secciones, y precisamente donde SoT se degrada [12]. Exige un Γ fuerte y auditoría de coherencia
Razonamiento matemático multisalto	Mala	Dependencias de resultado; aquí gana la descomposición secuencial [51]
Código con estado mutable compartido	Mala	El fallo canónico por decisiones implícitas en conflicto [33]
Chat interactivo de baja latencia	Muy mala	Los métodos sin pérdida de un solo nodo ya dominan [13, 14, 15]

11.2 V0 — el impuesto de coherencia

La implementación de referencia que acompaña a este artículo implementa V0 en su totalidad, sin red. Ejecuta la tubería completa en un solo proceso, barriendo S (reportado como p) y N , y comparando contra dos líneas base: la generación

monolítica con el mismo modelo, y la decodificación especulativa como comparador honesto de un solo nodo.

Reporta la puntuación de rejilla de entidades, la taxonomía de errores de costura, una puntuación global de juez mantenida aparte de ambas, la ρ alcanzada, la similitud y la vía por costura, y el impuesto de coherencia resultante.

Continuar o abandonar. Debe existir una ρ a la que la degradación de coherencia sea **inferior al 5 % respecto de la generación monolítica, en al menos una categoría de tarea**. Si no existe tal ρ , la arquitectura no es viable para el ensamblaje generativo, y el proyecto debería o bien detenerse o bien restringirse a cargas de trabajo sin costura que romper: clasificación, extracción, etiquetado. El banco de pruebas imprime este veredicto en cada ejecución.

La intención de enunciar una regla de parada antes de recoger datos es hacer que el resultado sea informativo en ambas direcciones.

11.3 Primeras mediciones

V0 y la calibración del acuerdo ya se han ejecutado contra modelos reales. Lo que sigue es el resultado completo, incluida la parte que no respalda una afirmación hecha antes en este mismo artículo.

Montaje. Tres familias de modelos servidas por un Ollama local —llama3.2:3b, qwen2.5:3b, gemma2:2b— con nomic-embed-text para los embeddings. Ocho prompts en ocho categorías, una semilla, temperatura 0. τ_{sem} se calibró sobre **72 pares etiquetados** ($F_{0.5}$ = 0,988, precisión 1,00, exhaustividad 0,944) y quedó en **0,51**. Los metadatos de la ejecución registran que no se usó el backend simulado y que la ruta de embeddings no degradó; sin ambas cosas los números de abajo son nulos, y por eso el banco de pruebas los reporta.

El impuesto de coherencia baja de forma monótona con ρ . Impuesto tipo BoookScore contra la línea base monolítica, con k = 1 y 21 celdas válidas por ρ :

ρ (objetivo)	ρ (alcanzada)	Impuesto de coherencia	Diferencia absoluta
1,00	1,17	+24,1 %	+0,124
1,25	1,27	+20,4 %	+0,076
1,50	1,53	+16,1 %	+0,068
2,00	2,08	+13,7 %	+0,052

Tanto la razón como la diferencia absoluta —que no depende del denominador— decrecen con ρ . Es el comportamiento que predice la hipótesis H1 y la primera evidencia de que el presupuesto de contexto de la sección 4 es la variable que el diseño dice que es.

El criterio de continuar o abandonar de la sección 11.2 se cumple. Seis de 28 celdas categoría $\times$ ρ quedan por debajo del umbral del 5 % fijado antes de que existiera dato alguno. synthetic_data lo cumple en todas las ρ probadas (+1,3 %, −5,1 %, −6,2 %, −0,3 %); creative_writing lo cumple con ρ = 2,0 y **−9,0 %**, y code_shared_state con ρ = 1,5 y +3,2 %. Un impuesto negativo significa que fragmentar *mejoró* la respuesta en ese instrumento. El criterio se redactó como «al menos una categoría de tarea» precisamente porque nadie esperaba que pasaran todas, y no pasan.

Dos fallos de medición, reportados porque acotan lo que la tabla anterior puede significar. El primero: la rejilla de entidades es inservible en este corpus. La línea base monolítica de ese instrumento va de 0,000 a 0,114 en los ocho prompts, con mediana 0,024. Las 96 celdas dividen, por tanto, por un denominador casi nulo, y las 96 quedan excluidas. El impuesto de coherencia por rejilla de entidades **no está medido aquí**, y una versión temprana de esta ejecución llegó a reportar cifras de hasta −180 % antes de que se revisara el denominador. El segundo: un prompt produjo una línea base monolítica de una sola frase y seis tokens —un fallo de generación, no un resultado de coherencia— y sus 12 celdas quedan excluidas de la tabla anterior.

La calibración del acuerdo no respalda el mapa de confianza. Barriendo $k \in \{1, 3, 5\}$ con ρ = 1,5 y una réplica por familia:

k	Impuesto de coherencia	Acuerdo medio	ALTA	BAJA
1	+13,2 %	—	—	—
3	+30,8 %	0,577	29,1 %	42,3 %
5	+33,3 %	0,728	58,3 %	28,7 %

El consenso por alineamiento múltiple cuesta entre 17 y 20 puntos de calidad respecto de k = 1, y la puntuación de acuerdo por unidad no predice la aceptabilidad juzgada: **r de Pearson = −0,030 sobre 597 unidades semánticas**. Los tramos de acuerdo son planos y no monótonos, y el acuerdo no los ordena: el tramo que más puntúa es el de 0,6–0,8, el de *menor* acuerdo queda segundo, y el tramo donde los modelos más coincidieron puntúa por debajo de ambos:

Acuerdo	Unidades	Juzgadas aceptables
0,0 - 0,2	40	97,5 %
0,2 - 0,4	91	91,2 %
0,4 - 0,6	80	91,3 %
0,6 - 0,8	122	99,2 %

0,8 - 1,0	264	91,3 %
-----------	-----	--------

La sección 11.4 enuncia que una correlación plana o negativa invalidaría el mapa de confianza como señal de fiabilidad y que tal resultado debe ser publicable. Queda publicado aquí.

Pero esta ejecución no es el experimento que especifica la sección 11.4, y la diferencia importa. El juez aceptó el **93,3 %** de las unidades. Con tan poca varianza en la variable dependiente, una correlación no puede aparecer aunque la señal subyacente exista, de modo que esta medición no distingue entre dos conclusiones muy distintas: que el acuerdo entre familias de modelos independientes no predice la corrección, o que un juez de clase par no discrimina la calidad con finura suficiente para detectarlo. V3c, tal como está especificado, pide conjuntos de datos con verdad de referencia; esta ejecución usó el juez. **La afirmación honesta es que el mapa de confianza está sin sustento, no refutado.**

Esa distinción no rescata la afirmación. Una propiedad sin sustento no puede anunciarse como la más valiosa de la arquitectura, y la sección 1.3 se ha reescrito en consecuencia. Sí significa que el mecanismo no está muerto todavía, y que el experimento que zanjaría la cuestión pasa a ser el punto de mayor prioridad de la sección 11.4.

Alcance. Ocho prompts, una semilla, modelos de 2-3 B. Es un corpus de prueba de humo, no un banco de referencia, y 2-3 B es el extremo bajo del rango de capacidad al que apunta el protocolo: un resultado aquí acota la arquitectura a esa escala y no la zanja a 8 B. Estos números son una señal sobre la que actuar, no un titular que citar.

11.4 Fases posteriores

V1 — router. Entrenar el clasificador de descomponibilidad con pérdida asimétrica; exigir la recuperación de ≥ 80 % de la ganancia disponible con una tasa de falsos positivos inferior al 5 %.

V2 — enjambre simulado. Inyectar rotación con parámetros medidos de voluntarios: ciclo de servicio 0,61, vida mediana del anfitrión 91 días [47, 48], distribuciones de latencia de área amplia [24], tasas de fallo del 5/10/20 %. Exigir una latencia p95 dentro de $2\times$ del caso sin fallos con $p = 0,10$, $N = 8$, usando peticiones especulativas.

V3 — red real, 20-50 nodos. Integrar el esquema de compromiso [68] y la auditoría muestreada [69]. Inyectar deliberadamente nodos deshonestos: modelos infradimensionados, fabricaciones plausibles, inyección de prompts. Exigir >95 % de detección con menos del 5 % de sobrecoste de verificación.

V3c — calibración del acuerdo. Medir la correlación entre la puntuación de acuerdo por unidad de la sección 8.4b y la corrección factual, contra conjuntos de datos con verdad de referencia, con réplicas tomadas de familias de modelo deliberadamente distintas. Reportar curvas de calibración por categoría de tarea. Hasta que este experimento se ejecute, las etiquetas de confianza se reportan como *acuerdo* y nunca como *exactitud*; una correlación plana o negativa invalidaría el mapa de confianza como señal de fiabilidad, y ese resultado debe ser publicable.

V4 — medición ambiental. Instrumentar y publicar el SCI frente a una línea base centralizada.

11.5 Métricas

Métrica	Definición	Objetivo v1.0	Medido (sección 11.3)
Impuesto de coherencia	Δ de la fracción de oraciones sin costura frente a monolítico	<5 %	cumplido en 3 de las 7 categorías que produjeron medición; 13,7 % global con $\rho = 2,0$
ρ operativa	Tokens de entrada por token de prompt	$<2,0$	sin medir todavía
Aceleración efectiva	Frente a monolítico, mismo modelo	$>1,5\times$	sin medir todavía
Aceleración frente a la línea base honesta	Frente a decodificación especulativa	Reportada incluso cuando es <1	sin medir todavía
Latencia p95 bajo rotación	$p=0,10$, $N=8$	$<2\times$ la del caso sin fallos	sin medir todavía
Detección de nodos deshonestos	Adversarios inyectados capturados	>95 %	sin medir todavía
Sobrecoste de verificación	Coste extra por fragmento	<5 %	sin medir todavía
SCI	gCO ₂ e por unidad funcional	Publicado y comparado	sin medir todavía

12. Limitaciones y resultados negativos

Enunciados sin rodeos y con extensión. Una especificación cuyos modos de fallo están documentados puede ser mejorada por gente que no la escribió; una que los oculta solo puede descubrirse errónea. Nada de lo que sigue retracta la sección 1.3: son las condiciones bajo las cuales esas posibilidades *no* se materializan, y constituyen la agenda del trabajo posterior a la publicación.

L1 — La pérdida de calidad por generación independiente es teórica, no incidental. La generación paralela

supone independencia condicional, y la calidad se degrada en proporción a la fuerza de las dependencias reales [20]. A igual presupuesto de cómputo, la descomposición es un canal con pérdidas [21]. Esto no se puede eliminar mediante ingeniería de prompts; solo se puede esquivar por encaminamiento, que es la razón por la que existe la sección 8.1.

L2 — La coherencia es el eje que se rompe, y las puntuaciones agregadas lo ocultan. SoT mejora la relevancia y la diversidad mientras degrada la coherencia y la inmersión [12]. El texto fusionado jerárquicamente exhibe ocho clases recurrentes y taxonomizables de error de coherencia [54]. Cualquier evaluación que reporte un único juicio de «cuál es mejor» pasará por alto el daño.

L3 — El ensamblador opera en un régimen que se sabe poco fiable. La fiabilidad de los modelos se degrada con la longitud de la entrada en todos los modelos probados; un distractor perjudica y cuatro se agravan; y los modelos rinden *mejor* con contextos barajados que con contextos lógicamente coherentes, lo que significa que un ensamblador que lee fragmentos semirrelacionados está mediblemente mermado [88]. El sesgo posicional añade una curva en U en la que los fragmentos centrales quedan sistemáticamente infraponderados [89].

L4 — El límite de contexto se reubica, no se elimina. Se traslada al cliente, que es el nodo más débil del sistema. El ensamblaje jerárquico hace que el requisito de memoria de trabajo crezca logarítmicamente y no linealmente con el volumen, lo que eleva sustancialmente el techo práctico; pero el techo existe, y está acotado tanto por el tiempo de ensamblaje como por la memoria.

L5 — Un orquestador de 8 B puede ser inadecuado. Véase la sección 5.4. Esto es H3, y un resultado negativo obligaría a un requisito mayor en el lado del cliente, lo que estrecha la base de usuarios alcanzable.

L6 — Sin resistencia fuerte a Sybil. Véase la sección 9.6.

L7 — La fragmentación no es cifrado. Véase la sección 9.1.

L8 — El beneficio ambiental no está probado. Véase la sección 10.3.

L9 — La analogía genómica es un vocabulario de diseño, no una herencia. Véase la sección 2.4. No se transfiere ningún algoritmo de ensamblaje de genomas, y los revisores procedentes de la bioinformática deberían leer la analogía como una convención de nomenclatura más una advertencia transferible sobre las repeticiones.

L10 — El riesgo dominante no es técnico. La computación voluntaria lleva dos décadas en declive: los primeros proyectos atrajeron del orden de un millón de voluntarios, y la base de usuarios se ha reducido desde entonces a unos doscientos mil [47]. Swarmby debe explicar qué hace distinto a su bucle de incentivos, y «créditos de red» no es por sí solo una respuesta. Esto es, según mi valoración, más probable que acabe con el proyecto que cualquier limitación algorítmica.

L11 — Los sistemas multiagente fallan de formas caracterizadas. Una taxonomía construida a partir de 150 trazas anotadas por expertos ($\kappa = 0,88$) y más de 1.600 trazas en siete marcos atribuye el 47,9 % de los fallos al diseño del sistema, el 32,2 % al desalineamiento entre agentes y el 20,0 % a la verificación de tareas, con la repetición de pasos (15,7 %) y la desobediencia de la especificación (11,8 %) como los modos individuales más comunes [90]. Swarmby es un sistema multiagente y debería esperar esta distribución.

L12 — Un enjambre de confianza reubica la confianza; no la elimina. Véase la sección 9.5. Quien controla la lista blanca de pertenencia controla el enjambre, de modo que la gobernanza del registro pasa a ser una función crítica de seguridad; el TLS mutuo autentica una identidad, no el modelo que hay detrás; y un enjambre que baja k a 1 renuncia al mapa de confianza de la sección 8.4b junto con la redundancia adversarial que ya no necesita. El nivel es un cambio de modelo de amenaza, y la especificación exige que se declare en lugar de darse por supuesto.

L13 — El mapa de confianza no tiene valor demostrado. Véase la sección 11.3. Medido contra un juez de clase par, el acuerdo por unidad no predijo la aceptabilidad juzgada ($r = -0,030$ sobre 597 unidades), y $k > 1$ costó entre 17 y 20 puntos de coherencia en la misma ejecución. El mecanismo está divulgado y especificado; su beneficio no está establecido, y la medición que lo establecería aún no se ha ejecutado. Quien construya sobre E16 debería tratarlo como una hipótesis sin validar.

L14 — El segundo instrumento de coherencia no funciona con respuestas cortas. Véase la sección 11.3. La rejilla de entidades devolvió líneas base monolíticas entre 0,000 y 0,114 en todo el corpus, lo que convierte cualquier comparación relativa construida sobre ella en una razón con denominador casi nulo. O el corpus de evaluación pasa a salidas más largas o se sustituye el instrumento; hasta entonces este artículo tiene un instrumento de coherencia en funcionamiento, no dos, y un único proxy mecánico es evidencia más delgada de lo que el diseño merece.

13. Declaración de arte previo

Publicado de forma defensiva para constituir arte previo. Los elementos que siguen se divulgan con la intención de que pasen al dominio público a efectos de patentabilidad; el autor se reserva el copyright del texto bajo CC BY 4.0 y licencia la implementación bajo AGPL-3.0-or-later. Cada elemento se divulga con detalle habilitante en la sección citada.

E1. Un método de inferencia distribuida de modelos de lenguaje en el que la unidad de distribución es una **subtarea semántica derivada de la petición**, despachada una vez por fragmento y por sesión a nodos que ejecutan cada uno un

modelo independiente completo, en lugar de una partición de los parámetros del modelo que exija travesía de red por token. (secciones 1.2 y 6.2)

E2. Un presupuesto de contexto S como parámetro explícito del protocolo que gobierna conjuntamente la coherencia del ensamblaje, la verificabilidad de los fragmentos, la fuga de privacidad y la capacidad exigida al trabajador, con la tasa de redundancia acompañante $\rho = \sum |K_i|/|P|$ como medida de coste reportada. (sección 4)

E3. Un contrato global Γ —objetivo, audiencia, registro, formato, longitud objetivo, tabla canónica de entidades, semilla de estilo— transmitido con cada fragmento como mecanismo de consistencia entre fragmentos, con la tabla de entidades sirviendo de sistema de coordenadas de nomenclatura compartido. (sección 7.2)

E4. Un router con coste de decisión asimétrico que puede negarse a fragmentar, calibrado con F_β donde $\beta < 1$ de modo que la fragmentación errónea se penalice por encima de la negativa errónea. (sección 8.1)

E5. Un planificador de DAG de dependencias que distingue las tareas que requieren el *resultado* de un predecesor de las que requieren solo su *enunciado*, con el paralelismo acotado por la anchura del nivel y con negativa ante planes degenerados. (sección 8.2)

E6. Un procedimiento de empaquetado en el que el contrato global nunca se elide para ajustarse a un presupuesto de contexto, y en el que el déficit de presupuesto se resuelve reduciendo el número de fragmentos en lugar del contexto compartido. (sección 8.3)

E7. Un ensamblador de seleccionar y empalmar en el que los múltiples fragmentos candidatos se resuelven por selección y no por síntesis, con el puenteo generativo invocado únicamente en una costura cuya similitud de frontera cae por debajo de un umbral calibrado, y con cada costura y su vía registradas. (sección 8.4)

E8. Calibración empírica del umbral de costura a partir de pares etiquetados de costura y no-costura bajo un objetivo asimétrico, rederivada por modelo de embeddings, en lugar de una constante coseno fija. (sección 8.5)

E9. Una auditoría de coherencia devuelta como parte de la respuesta del protocolo, que combina la coherencia local por rejilla de entidades con una taxonomía de errores de costura mecánicamente detectable, reportada por separado de cualquier puntuación de calidad agregada. (sección 8.6)

E10. Encaminamiento por carriles de sensibilidad —PUBLIC / SANITISABLE / SENSITIVE— como mecanismo de confidencialidad, con el carril sensible ligado a la ejecución local o a la ejecución confiable atestiguada, en lugar de cualquier afirmación de que la fragmentación proporciona confidencialidad. (sección 9.4)

E11. Un esquema de verificación de dos capas para trabajadores no confiables que combina un compromiso sensible a la localidad sobre las activaciones ligado a un perfil de nodo declarado, con auditoría pública muestreada indistinguible del trabajo real, y redundancia aplicada según la criticidad del fragmento en vez de uniformemente. (secciones 9.3, 7.4 y 7.5)

E12. Despacho redundante que preserva la diversidad: las réplicas de un fragmento crítico se asignan a nodos de familias de modelos deliberadamente *distintas*, sobre la base de que la calidad de la selección depende de la diversidad de los candidatos, mientras las clases de capacidad se mantienen homogéneas dentro del papel de un fragmento. (secciones 5.3(c) y 7.6)

E13. Despacho especulativo con disparo en el p95 de latencia por clase con una contabilidad de cancelaciones que distingue en la reputación los nodos lentos de los deshonestos. (secciones 7.6 y 6.3)

E14. Una unidad de contabilidad de crédito no transferible, no preminado y con caducidad ganada por el procesamiento verificado de fragmentos y gastada en peticiones, con conversión unidireccional a moneda fiduciaria a través de una capa comercial de servicio. (sección 10.1)

E15. La descomposición cobertura/conversión $Q \approx V \cdot \Pi_i C_i$ como instrumento de diseño para la inferencia en enjambre, con el corolario de que los nodos adicionales elevan la cobertura con retornos decrecientes mientras que la conversión está acotada por el selector del lado del cliente. (secciones 5.2 y 5.3)

E16. Consenso por alineamiento múltiple de réplicas generadas de forma independiente, en el que k respuestas completas a la misma microtarea, producidas por nodos de familias de modelo deliberadamente distintas, se alinean con granularidad de unidad semántica para producir una salida de consenso junto con una **puntuación de acuerdo por unidad**, y en el que las unidades por debajo de un umbral de acuerdo se exponen al usuario como regiones de baja confianza, siendo el mapa de fiabilidad un producto directo de la redundancia que la descentralización exige, y análogo a la calidad por base de un ensamblaje de genoma. Se divulga como **mecanismo**: su correlación con la corrección se midió y salió plana (sección 11.3), de modo que aquí no se le atribuye ningún beneficio de fiabilidad. (secciones 8.4b y 11.3)

E17. Un modelo de cobertura para el ensamblaje semántico en el que el plan previo a la generación y el contrato global sirven de secuencia de referencia, la pérdida de paquetes —y no la colocación de las muestras— es el elemento estocástico, y el requisito de redundancia se deriva como $c \geq \ln(1/\epsilon)/(1-p)$ a partir de una tolerancia declarada ϵ de unidades semánticas sin cubrir a la tasa de pérdida de nodos medida p . (sección 5.4.1)

E18. Escalonamiento dinámico de privacidad con enjambres de confianza: un clasificador de privacidad del lado del cliente —bandera manual dura con precedencia, más triaje local de entidades nombradas orientado a la exhaustividad que nunca abandona la máquina— que gobierna una decisión de enrutamiento sobre un eje ortogonal a la

sensibilidad del contenido, hacia una malla global no confiable, hacia una submalla permisionada cuya pertenencia es una lista blanca criptográfica de claves públicas bajo TLS mutuo y un registro controlado por el operador, o hacia ejecución puramente local; con el mismo protocolo y el mismo cliente en los tres casos, con la holgura de latencia de la submalla permisionada gastada en el presupuesto de contexto y no en una partición de grano más fino, y con la reducción del número de réplicas dentro de un enjambre de confianza permitida sólo bajo declaración explícita de que no se produjo mapa de confianza. (sección 9.5)

14. Conclusión

El conocimiento necesario para construir modelos de lenguaje es público. El capital necesario para operarlos no lo es, y esa asimetría —y no ningún secreto— es lo que concentra el control sobre una tecnología de propósito general. Entretanto, el hardware capaz de servir inferencia está ocioso en cientos de millones de hogares y oficinas, ya fabricado, ya consumiendo energía.

El obstáculo entre esos dos hechos es físico y específico: cuatro a cinco órdenes de magnitud entre la interconexión de centro de datos y los enlaces domésticos, que todo diseño existente de inferencia entre pares cruza en cada token generado. La afirmación estructural de este artículo es que cruzarlo **una vez por unidad de trabajo en lugar de una vez por token** es un régimen distinto y no una optimización del mismo, y que eso es lo que hace posible la participación voluntaria en absoluto.

Se siguen tres consecuencias que no anticipaba cuando comenzó este trabajo y que considero las aportaciones sustantivas del artículo. El **presupuesto de contexto** unifica cuatro propiedades de diseño —coherencia, verificabilidad, privacidad y capacidad exigida al trabajador— como funciones de un único escalar, lo que convierte la viabilidad del protocolo en una sola proposición falsable en vez de en cuatro argumentos. El **modelo de cobertura** convierte el marco genómico en una derivación al situar la aleatoriedad donde los supuestos clásicos sí se cumplen: en qué paquetes vuelven, no en dónde caen las muestras. Y el **consenso por alineamiento múltiple** produce un mapa de fiabilidad que la inferencia centralizada no puede generar, porque la redundancia que la descentralización exige es precisamente lo que pone estimadores independientes a disposición de la comparación. Una cuarta llegó más tarde y es menor en teoría pero mayor en consecuencias: el **escalonamiento de privacidad**, bajo el cual el mismo protocolo se ejecuta sobre una malla abierta, sobre una lista blanca permisionada con TLS mutuo, o sobre nada más que la máquina local — porque una arquitectura que no puede operarse lícitamente sobre datos regulados no es alternativa de nada, por buenos que sean sus números.

El argumento en contra es igual de específico y se enuncia extensamente en la sección 12. La generación independiente pierde calidad por razones teóricas y no incidentales. El modelo del lado del cliente del que depende el diseño se sitúa en el extremo débil de la capacidad de planificación medida. La computación voluntaria lleva veinte años contrayéndose, y ningún diseño de incentivos de este artículo es todavía una respuesta demostrada a por qué eso habría de revertirse.

Ambos argumentos son reales, y ninguno se zanja discutiendo. Lo que lo zanja es una medición. La primera ya está: contra tres familias de modelos locales el impuesto de coherencia decrece de forma monótona al subir el presupuesto de contexto, y el umbral de abandono fijado de antemano se supera en tres categorías de tarea, mientras que el mapa de confianza —la propiedad de la que este artículo estaba más orgulloso— volvió sin relación medible con la calidad. Lo que queda por zanjar es si existe un presupuesto de contexto que satisfaga coherencia, privacidad, verificabilidad y capacidad del trabajador simultáneamente, a un coste inferior al valor de la capacidad agregada. He especificado el protocolo con detalle suficiente para implementarlo, enunciado las hipótesis que lo falsarían, publicado el banco de pruebas que mide la primera de ellas y comprometido de antemano el umbral a partir del cual concluiría que el diseño no funciona.

Si funciona, el resultado no es una forma más barata de comprar lo que ya se vende. Es capacidad de inferencia que crece con el número de personas que participan y no con la cantidad de capital disponible para construir, sostenida bajo una licencia y una estructura de gobernanza diseñadas para que ninguna parte pueda cercarla. Eso merece intentarse incluso con una probabilidad sustancial de fracaso, y merece intentarse en abierto, donde pueda comprobarse.

Referencias

Estado de verificación y estilo de cita. La cita en el texto emplea los marcadores numéricos [1]...[90]; la numeración no ha cambiado respecto de versiones anteriores de este documento. La lista de referencias que sigue está formateada en **APA 7.ª edición**. Los identificadores marcados con  no pudieron confirmarse contra una fuente primaria durante la preparación y **deben completarse o verificarse antes del envío**; esas entradas se dejan deliberadamente incompletas a la vista en lugar de rellenarse de memoria. Las entradas cuya lista de autores aparece abreviada como «et al.» arrastran la atribución registrada en el borrador anterior y deben ampliarse a la lista completa de autores exigida por APA antes del envío. Todo lo no marcado se comprobó contra la página del editor o la página de resumen de arXiv. Las anotaciones completas, incluidos los hallazgos cuantitativos, están en la bibliografía del proyecto (docs/REFERENCES.md).

Inferencia y entrenamiento descentralizados

- [1] Borzunov, A., et al. (2023). Petals: Collaborative inference and fine-tuning of large models. *Association for Computational Linguistics (ACL) 2023: System Demonstrations*. <https://arxiv.org/abs/2209.01188>
- [2] Borzunov, A., et al. (2023). Distributed inference and fine-tuning of large language models over the internet. *arXiv*. <https://arxiv.org/abs/2312.08361>
- [3] Proyecto Petals. (2023). *Repositorio del proyecto Petals, versión v2.2.0* [Software].
- [4] Ryabinin, M., & Gusev, A. (2020). Towards crowdsourced training of large neural networks using decentralized mixture-of-experts. *Advances in Neural Information Processing Systems*, 33, 3659–3672.
- [5] Ryabinin, M., Dettmers, T., Diskin, M., & Borzunov, A. (2023). SWARM parallelism: Training large models can be surprisingly communication-efficient. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 29633–29654.
- [6] Bittensor. (s. f.). *Incentivizing intelligence*. Bittensor. <https://bittensor.com/academia>
- [7] *Análisis de la concentración de participación en las subredes de Bittensor*. (s. f.). ⚠ Autoría, año y publicación sin verificar — completar antes del envío.
- [8] Douillard, A., et al. (2023). DiLoCo: Distributed low-communication training of language models. *arXiv*. <https://arxiv.org/abs/2311.08105>
- [9] Jaghouar, S., et al. (2024). OpenDiLoCo. *arXiv*. <https://arxiv.org/abs/2407.07852>
- [10] Jaghouar, S., et al. (2024). *INTELLECT-1 technical report*. *arXiv*. <https://arxiv.org/abs/2412.01152>
- [11] *Protocol/Subspace Networks*. (s. f.). ⚠ Autoría, año, título actual e identificador sin verificar — completar antes del envío.

Decodificación paralela y descomposición

- [12] Ning, X., Lin, Z., Zhou, Z., Wang, Z., Yang, H., & Wang, Y. (2024). Skeleton-of-thought: Prompting LLMs for efficient parallel generation. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2307.15337> (Incluye SoT-R.)
- [13] Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2211.17192>
- [14] Cai, T., et al. (2024). Medusa. *arXiv*. <https://arxiv.org/abs/2401.10774>
- [15] Fu, Y., et al. (2024). Break the sequential dependency of LLM inference using lookahead decoding. *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2402.02057>
- [16] Liu, M., et al. (2024). APAR: LLMs can do auto-parallel auto-regressive decoding. *arXiv*. <https://arxiv.org/abs/2401.06761>
- [17] Jin, T., et al. (2025). Learning to keep a promise (PASTA). *International Conference on Machine Learning (ICML)*. <https://arxiv.org/abs/2502.11517>
- [18] Jin, S., Wu, Y., Zheng, H., Zhang, Q., & Lentz, M. (2024). Adaptive skeleton graph decoding. *arXiv*. <https://arxiv.org/abs/2402.12280>
- [19] Rodionov, G., et al. (2025). Hogwild! Inference. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2504.06261>
- [20] Kang, W., Galim, K., Oh, S., et al. (2026). ParallelBench: Understanding the trade-offs of parallel decoding in diffusion LLMs. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2510.04767>
- [21] Tran, H., & Kiela, D. (2026). Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets. *arXiv*. <https://arxiv.org/abs/2604.02460>
- [51] Zhou, D., et al. (2023). Least-to-most prompting enables complex reasoning in large language models. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2205.10625>
- [52] Jiang, Z., et al. (2024). LongRAG. *arXiv*. <https://arxiv.org/abs/2406.15319>

Red y hardware

- [22] NVIDIA. (s. f.). *Documentación de producto de la NVIDIA H100* (NVLink 900 GB/s SXM; PCIe Gen5 128 GB/s). NVIDIA Corporation.
- [23] NVIDIA. (s. f.). *Documentación de NVIDIA Quantum-2 InfiniBand* (400 Gb/s por puerto; 51,2 Tb/s agregados). NVIDIA Corporation.
- [24] Sevilla, J. (2025). *How far can decentralized training over the internet scale?* Epoch AI. [Léase junto con las estadísticas de latencia entre regiones de Microsoft Azure.]
- [25] *Análisis de esquemas de paralelismo de modelo a latencias de internet pública*. (s. f.). ⚠ Autoría, año, título y

publicación sin verificar — completar antes del envío.

Ensamblaje de genomas

- [26] Lander, E. S., & Waterman, M. S. (1988). Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics*, 2(2), 231–239. [https://doi.org/10.1016/0888-7543\(88\)90007-9](https://doi.org/10.1016/0888-7543(88)90007-9)
- [27] Khadiev, K., & Safina, L. (2024). Quantum algorithms for the shortest common superstring and text assembling problems. *Quantum Information and Computation*, 24(3–4), 267–294. <https://doi.org/10.26421/OIC24.3-4-2>
- [28] *Revisión del ensamblaje de genomas distribuido y de alto rendimiento*. (s. f.). △ Autoría, año, título y publicación sin verificar — completar antes del envío.
- [29] Pevzner, P. A., Tang, H., & Waterman, M. S. (2001). An Eulerian path approach to DNA fragment assembly. *Proceedings of the National Academy of Sciences*, 98(17), 9748–9753.
- [30] Nagarajan, N., & Pop, M. (2013). Sequence assembly demystified. *Nature Reviews Genetics*, 14, 157–167. <https://doi.org/10.1038/nrg3367>
- [31] Kingsford, C., Schatz, M. C., & Pop, M. (2010). Assembly complexity of prokaryotic genomes using short reads. *BMC Bioinformatics*.
- [32] Chaisson, M. J. P., Wilson, R. K., & Eichler, E. E. (2015). Genetic variation and the de novo assembly of human genomes. *Nature Reviews Genetics*.

Sistemas multiagente, selección y agregación

- [33] Yan, W. (2025). *Don't build multi-agents*. Blog de ingeniería de Cognition.
- [38] Brown, B., et al. (2024). Large language monkeys: Scaling inference compute with repeated sampling. *arXiv*. <https://arxiv.org/abs/2407.21787>
- [39] Maryansky, A., Budnikov, D., & Kaliyev, A. T. (2026). When agents disagree: The selection bottleneck in multi-agent LLM pipelines. *arXiv*. <https://arxiv.org/abs/2603.20324>
- [40] Żywot, A., Chen, Y., Yuan, S., Søgaard, A., & de Rijke, M. (2026). Can small agents collaborate to beat a single large language model? *arXiv*. <https://arxiv.org/abs/2601.11327>
- [41] Wang, J., et al. (2025). Mixture-of-agents enhances large language model capabilities. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2406.04692>
- [42] Chen, Y., Niu, G., Cheng, J., Han, B., & Sugiyama, M. (2025). When and why does multi-agent debate fail, and does it really underperform? *arXiv*. <https://arxiv.org/abs/2510.20963>
- [90] Cemri, M., et al. (2025). Why do multi-agent LLM systems fail? *arXiv*. <https://arxiv.org/abs/2503.13657>
- [71] *AgenTracer*. (2025). *arXiv*. <https://arxiv.org/abs/2509.03312> △ Autoría sin verificar — completar antes del envío.

Modelos pequeños, planificación y encaminamiento

- [43] Belcak, P., et al. (2025). Small language models are the future of agentic AI. *arXiv*. <https://arxiv.org/abs/2506.02153> (Artículo de posición.)
- [44] Schepanowski, C., & Ling, C. (2025). On the limits of innate planning in large language models. *arXiv*. <https://arxiv.org/abs/2511.21591>
- [45] Valmeekam, K., et al. (2022). PlanBench. *arXiv*. <https://arxiv.org/abs/2206.10498>
- [46] Ong, I., et al. (2024). RouteLLM. *arXiv*. <https://arxiv.org/abs/2406.18665>

Embeddings y coherencia

- [34] Ethayarajh, K. (2019). How contextual are contextualized word representations? *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/1909.00512>
- [35] Steck, H., et al. (2024). Is cosine-similarity of embeddings really about similarity? *Companion Proceedings of the ACM Web Conference (WWW '24)*. <https://arxiv.org/abs/2403.05440>
- [36] Muennighoff, N., et al. (2023). MTEB: Massive text embedding benchmark. *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*. <https://arxiv.org/abs/2210.07316>
- [37] Sentence-Transformers. (s. f.). *Similitud semántica y minería de paráfrasis* [Documentación]. Sentence-Transformers.
- [53] Barzilay, R., & Lapata, M. (2008). Modeling local coherence: An entity-based approach. *Computational Linguistics*, 34(1).
- [54] Chang, Y., et al. (2024). BoookScore. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.00785>
- [88] Chroma. (2025). *Context rot* [Informe técnico]. Chroma.

[89] Liu, N. F., et al. (2023). Lost in the middle. *Transactions of the Association for Computational Linguistics (TACL)*. <https://arxiv.org/abs/2307.03172>

Verificación, privacidad y seguridad

[50] Zhang, Y., Wang, S., Liu, X., Tan, S., Popa, R. A., & Moallemi, C. C. (2024). Proof of sampling: A Nash equilibrium-secured verification protocol for decentralized systems. *arXiv*. <https://arxiv.org/abs/2405.00295>

[55] Sweeney, L. (2002). k-Anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5), 557–570.

[56] Machanavajjhala, A., et al. (2007). ℓ -Diversity: Privacy beyond k-anonymity. *ACM Transactions on Knowledge Discovery from Data*, 1(1).

[57] Narayanan, A., & Shmatikov, V. (2008). Robust de-anonymization of large sparse datasets. *IEEE Symposium on Security and Privacy (S&P)*.

[58] Narayanan, A., et al. (2012). On the feasibility of internet-scale author identification. *IEEE Symposium on Security and Privacy (S&P)*.

[59] *Cross-domain authorship attribution*. (2016). *Privacy Enhancing Technologies Symposium (PETS)*. △ Autoría sin verificar — completar antes del envío.

[60] *Forensic authorship analysis of microblogging texts*. (2020). *arXiv*. <https://arxiv.org/abs/2003.11545> △ Autoría sin verificar — completar antes del envío.

[61] Morris, J. X., et al. (2023). Text embeddings reveal (almost) as much as text. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2310.06816>

[62] Zhang, C., et al. (2024). Extracting prompts by inverting LLM outputs. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

[63] Fan, M., Liu, Y., Wang, F., & Chen, C. (2026). What does the server see? Understanding privacy leakage from large language models in split inference. *arXiv*. <https://arxiv.org/abs/2605.23158>

[64] Keller, M. (2020). MP-SPDZ: A versatile framework for multi-party computation. *ACM Conference on Computer and Communications Security (CCS)*.

[65] Hao, M., et al. (2022). Iron: Private inference on transformers. *Advances in Neural Information Processing Systems (NeurIPS)*.

[66] Lu, W., et al. (2025). BumbleBee: Secure two-party inference framework for large transformers. *Network and Distributed System Security Symposium (NDSS)*.

[67] Sun, H., Li, J., & Zhang, H. (2024). zkLLM: Zero knowledge proofs for large language models. *ACM Conference on Computer and Communications Security (CCS)*. <https://arxiv.org/abs/2404.16109>

[68] Ong, J., et al. (s. f.). *TOPLOC: A locality sensitive hashing scheme for trustless verifiable inference*. △ Año, publicación e identificador sin verificar — completar antes del envío.

[69] *VeriLLM: A lightweight framework for publicly verifiable decentralized inference*. (2025). *arXiv*. <https://arxiv.org/abs/2509.24257> △ Autoría sin verificar — completar antes del envío.

[70] OWASP Foundation. (2025). *OWASP top 10 for LLM applications*. OWASP Foundation.

[72] *Confidential computing on NVIDIA Hopper GPUs: A performance benchmark study*. (s. f.). △ Autoría, año e identificador sin verificar — completar antes del envío.

[73] *Benchmarking confidential GPU inference on NVIDIA H100 under Intel TDX*. (s. f.). △ Autoría, año e identificador sin verificar — completar antes del envío.

[74] Lukas, N., et al. (2023). Analyzing leakage of personally identifiable information in language models. *IEEE Symposium on Security and Privacy (S&P)*.

[75] *Differentially-private text generation degrades output language quality*. (2025). *arXiv*. <https://arxiv.org/abs/2509.11176> △ Autoría sin verificar — completar antes del envío.

[76] Douceur, J. R. (2002). The Sybil attack. *International Workshop on Peer-to-Peer Systems (IPTPS)*. https://doi.org/10.1007/3-540-45748-8_24

[77] Kamvar, S. D., Schlosser, M. T., & Garcia-Molina, H. (2003). The EigenTrust algorithm for reputation management in P2P networks. *International World Wide Web Conference (WWW)*.

Computación voluntaria

[47] Anderson, D. P. (2019). BOINC: A platform for volunteer computing. *Journal of Grid Computing*. <https://arxiv.org/abs/1903.01699>

[48] Anderson, D. P., & Fedak, G. (2006). The computational and storage potential of volunteer computing. *IEEE International Symposium on Cluster Computing and the Grid (CCGrid)*.

[49] *Idle consumer GPUs versus enterprise GPUs for LLM inference*. (2025). ACM AIBC. [△](#) Autoría e identificador sin verificar — completar antes del envío.

Licencias, gobernanza y energía

[78] Autoridad Federal Suiza de Supervisión de los Mercados Financieros. (s. f.). *Guidelines for enquiries regarding the regulatory framework for initial coin offerings*. FINMA.

[79] Parlamento Europeo & Consejo de la Unión Europea. (2023). *Regulation (EU) 2023/1114 on Markets in Crypto-Assets (MiCA)*.

[80] Free Software Foundation. (2007). *GNU Affero General Public License, version 3* (cláusula 13). Free Software Foundation.

[81] *Recambio de licencia de Redis a AGPLv3 (mayo de 2025); Elastic añade AGPLv3 (agosto de 2024)*. (2024–2025). [△](#) Autoría, editor y documentos fuente sin verificar — completar antes del envío.

[82] *Análisis comparativo de la oleada de recambios de licencia de 2021–2025*. (s. f.). [△](#) Fuente secundaria; autoría, año y publicación sin verificar — completar antes del envío.

[83] International Energy Agency. (2025). *Energy and AI*. International Energy Agency.

[84] *Estudio a nivel de instalación de la intensidad de carbono de la red eléctrica de los centros de datos hiperescala de EE. UU.* (2026). [△](#) Autoría, publicación e identificador sin verificar — completar antes del envío.

[85] *Banco de pruebas de inferencia de LLM con conciencia energética*. (2026). [△](#) Autoría, publicación e identificador sin verificar — completar antes del envío.

[86] Uptime Institute. (2025). *Global Data Center Survey 2025*. Uptime Institute.

[87] Green Software Foundation. (2024). *Software Carbon Intensity (SCI) specification* (ISO/IEC 21031:2024). Green Software Foundation.

Versión del documento 1.4, 14 de agosto de 2026. Documento complementario en inglés: WHITEPAPER_EN.md. Especificación del protocolo: SPEC_ES.md. Implementación de referencia y banco de pruebas del impuesto de coherencia: swarmbly_v0/. Registro público fechado: Zenodo 10.5281/zenodo.21956743, 10.5281/zenodo.21957088, y este repositorio en el tag v1.